



WEEK 3

SOFTWARE DEVELOPMENT TOOLS AND ENVIRONMENTS



Presented by

Asst. Prof. Dr. Tuchsana Ploysuwan



Week 3 Branches and Working with Others

● Week 3 Topics:

- Master/Main Branch and Branches
- Understanding HEAD
- Git Branch Commands:
 - **git branch, git switch, git checkout**
- Delete or Rename Branch
- Merging Branches and Conflicts
- Using **git diff**
- Exercise and Solution

Let's get started!

Week 3

Branches

Week 3 Branches and Working with Others

- Let's review what our current commit process looks like...

Week 3 Branches and Working with Others

● Commit Process

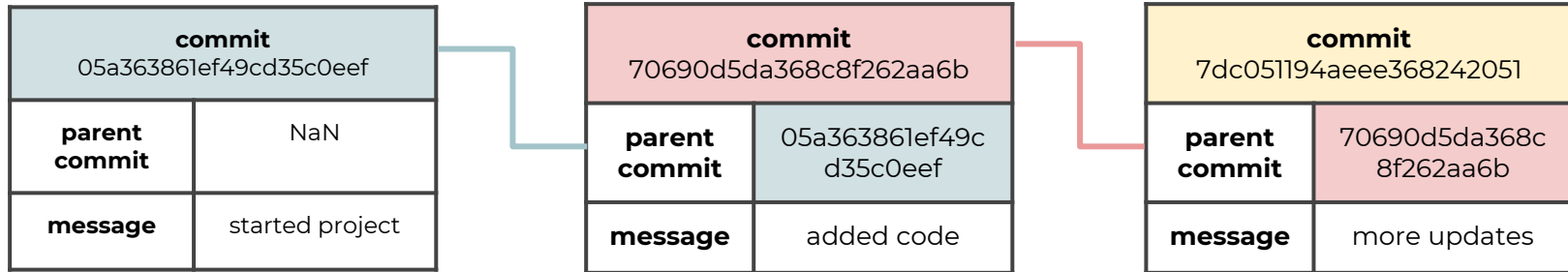
- As we create commits, we are linking to a parent commit, showing the log of the commit history.

commit 05a363861ef49cd35c0eef	
parent commit	NaN
message	started project

Week 3 Branches and Working with Others

● Commit Process

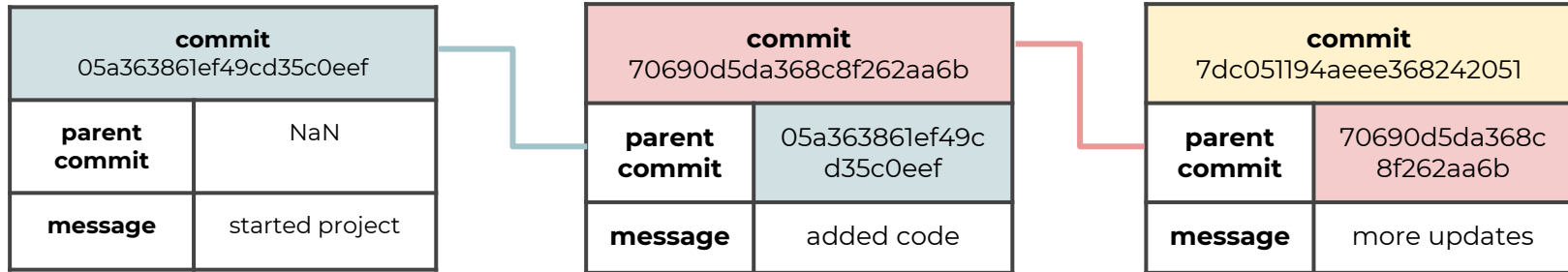
- As we create commits, we are linking to a parent commit, showing the log of the commit history.



Week 3 Branches and Working with Others

● Commit Process

- As we need incorporate the workflows of others or be able to focus on new updates without breaking old code, we need **branches**.



Week 3 Branches and Working with Others

● **Branches**

- A branch represents an independent line of development.
- Branches serve as an abstraction for the edit/stage/commit process.
- They are a way to request a brand new working directory, staging area, and project history.

Week 3 Branches and Working with Others

● **Branches**

- Branches are just pointers to commits.
- When you create a branch, all Git needs to do is create a new pointer, it doesn't change the repository in any other way.
- Let's explore why branches are useful for workflows...

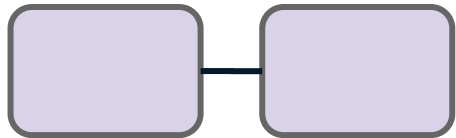
Week 3 Branches and Working with Others

- **Branches**



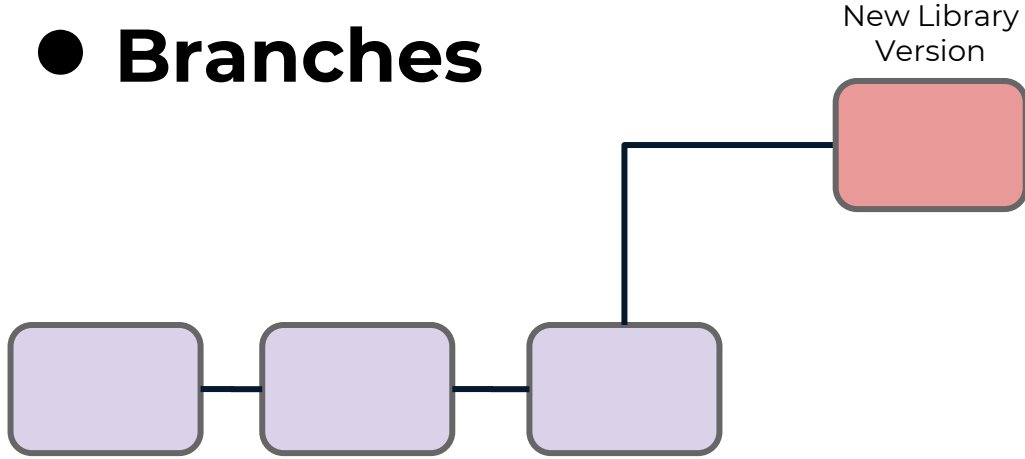
Week 3 Branches and Working with Others

- **Branches**



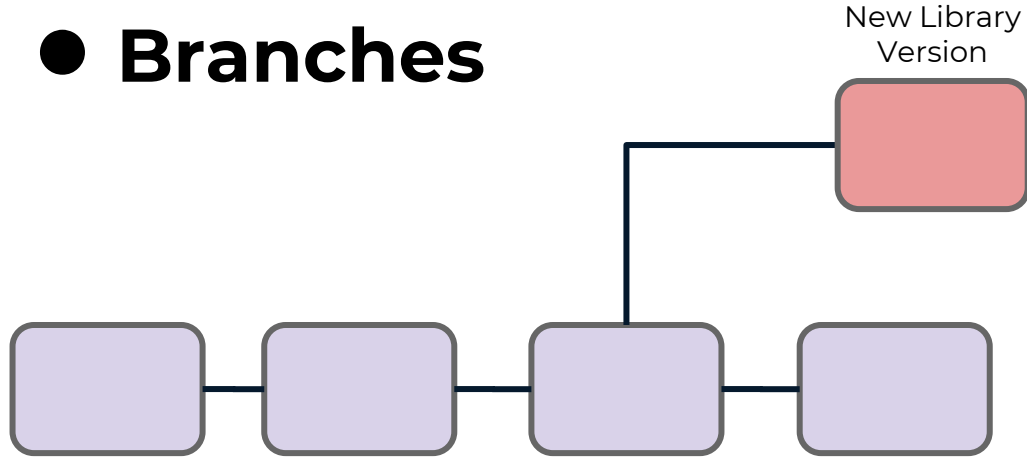
Week 3 Branches and Working with Others

● Branches



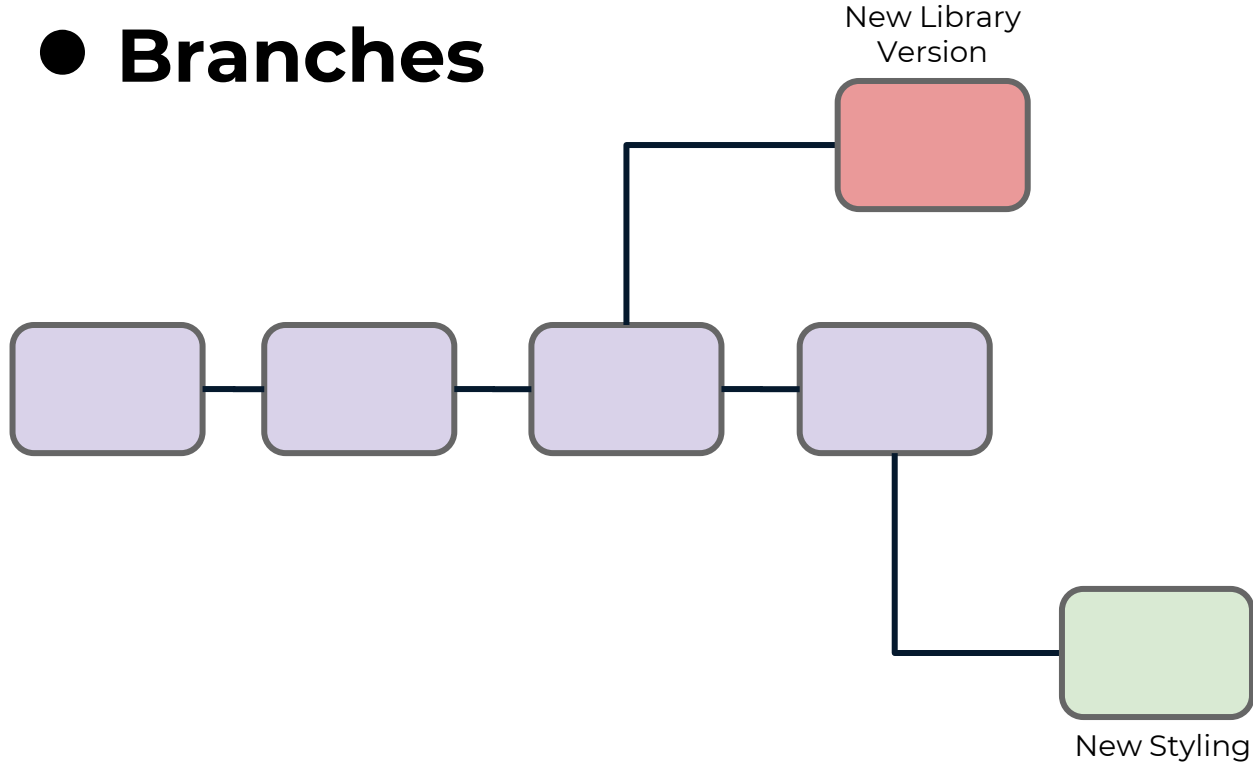
Week 3 Branches and Working with Others

● Branches



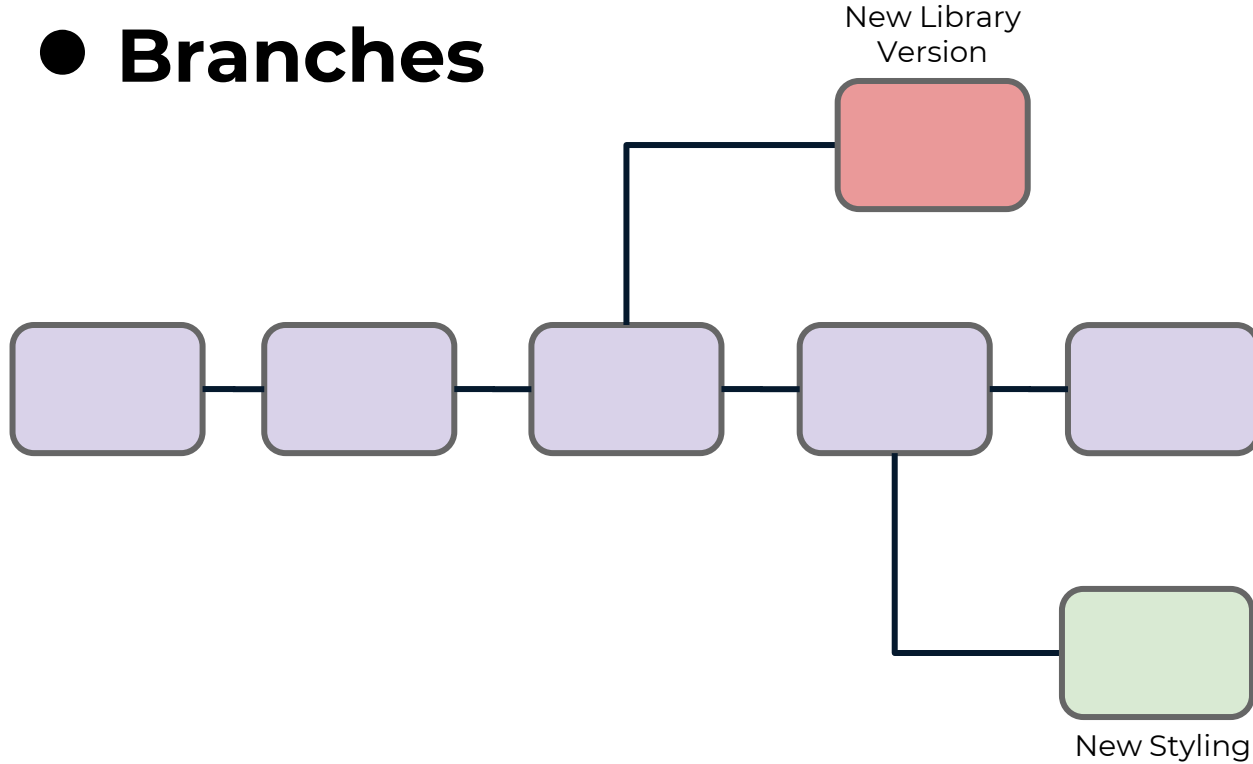
Week 3 Branches and Working with Others

● Branches



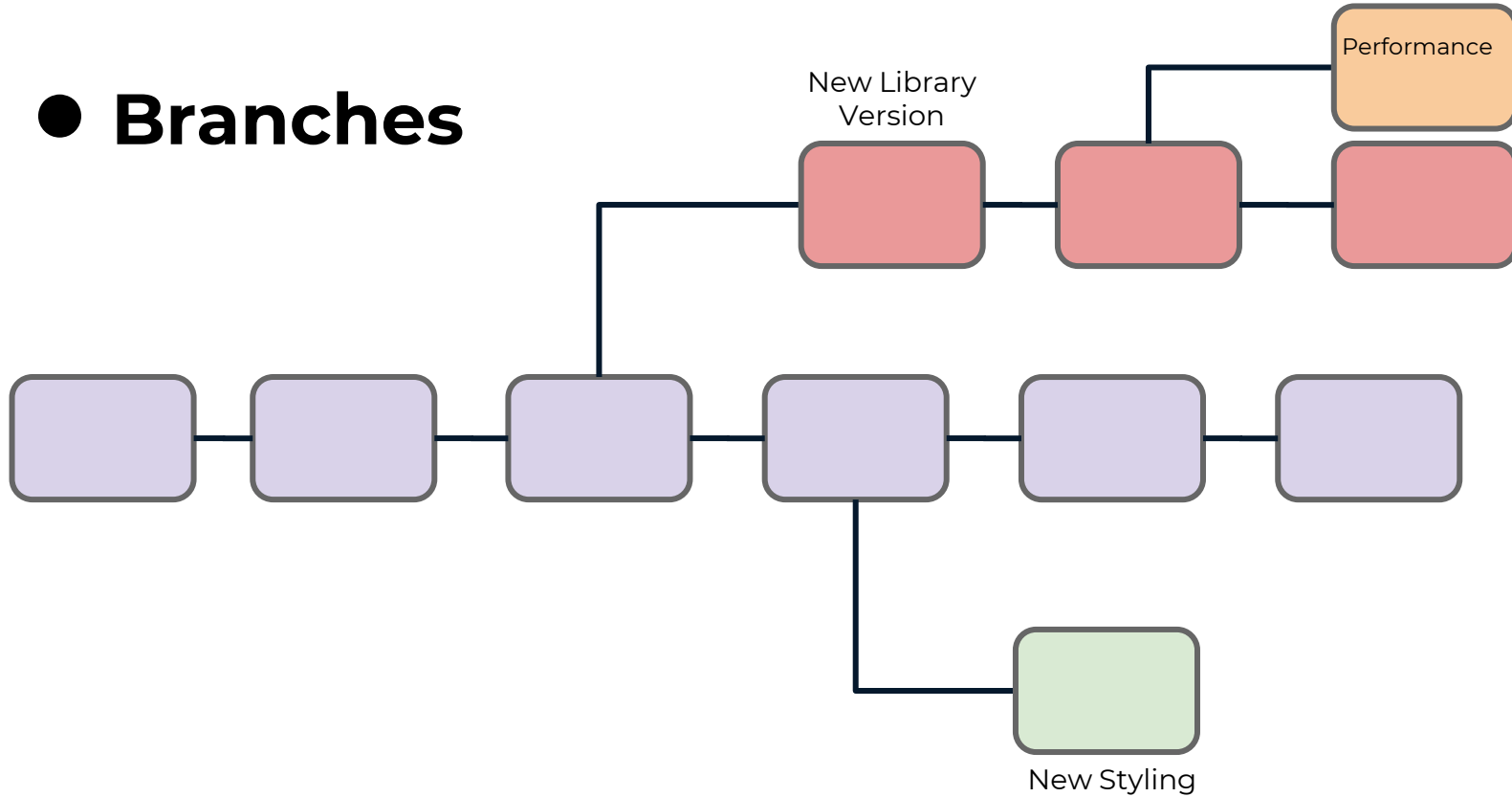
Week 3 Branches and Working with Others

● Branches



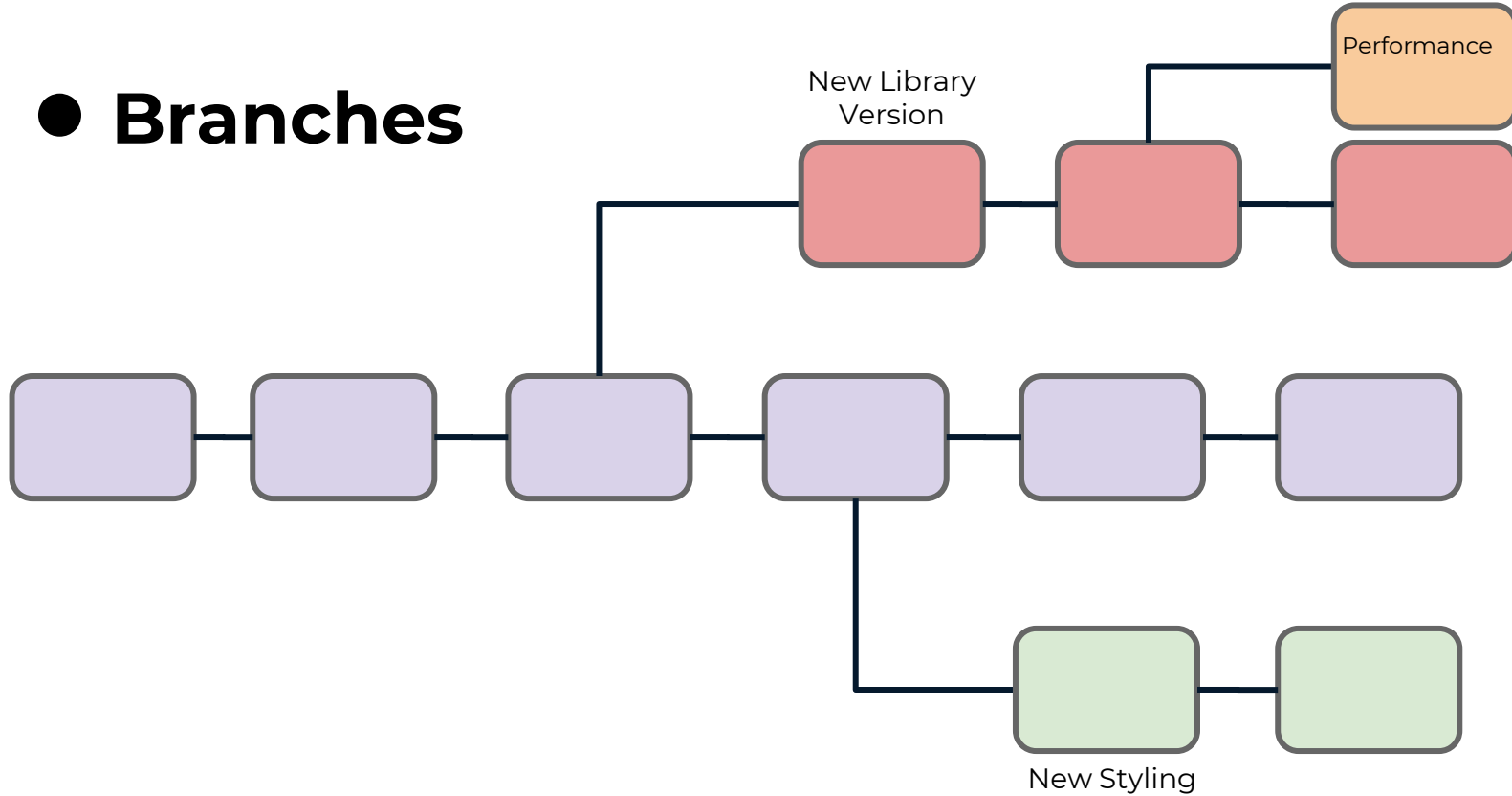
Week 3 Branches and Working with Others

● Branches



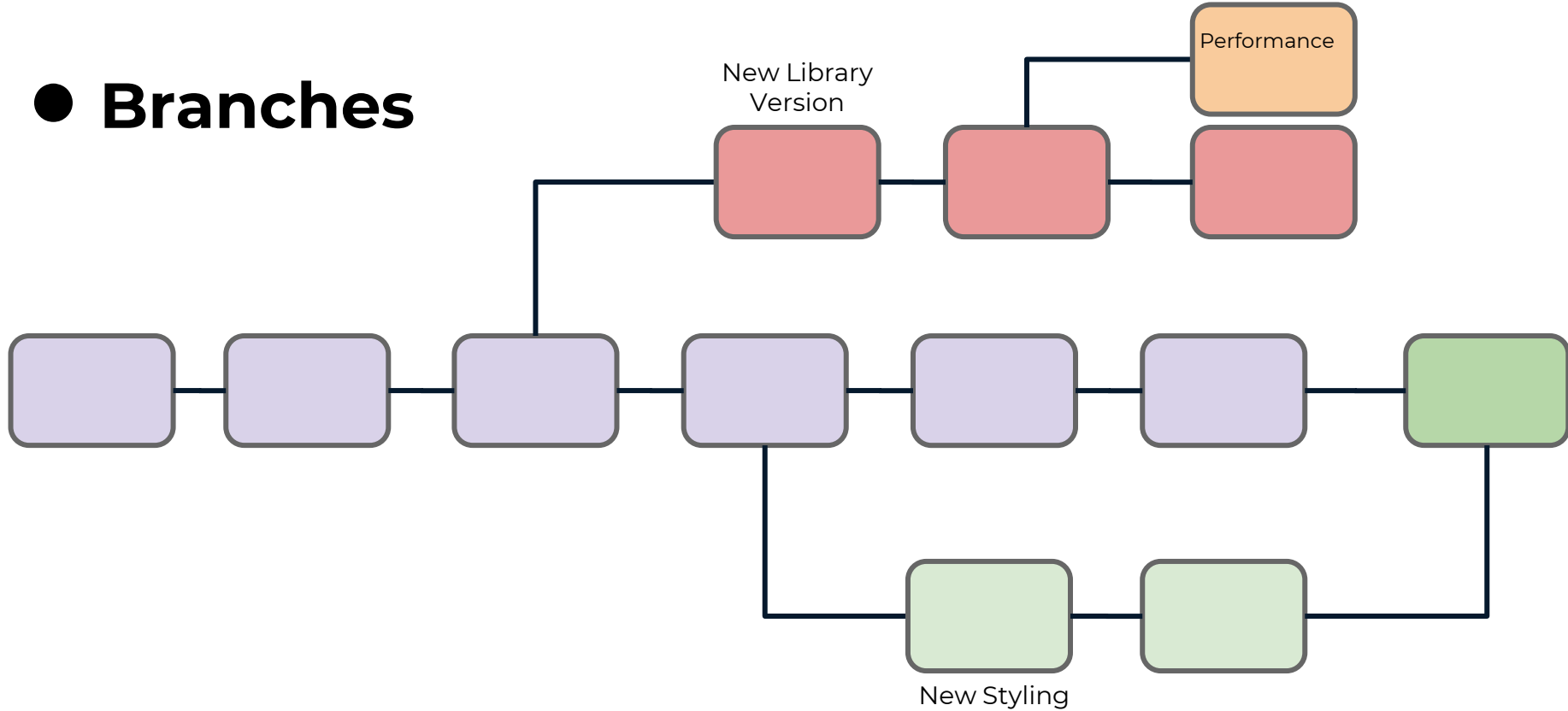
Week 3 Branches and Working with Others

● Branches



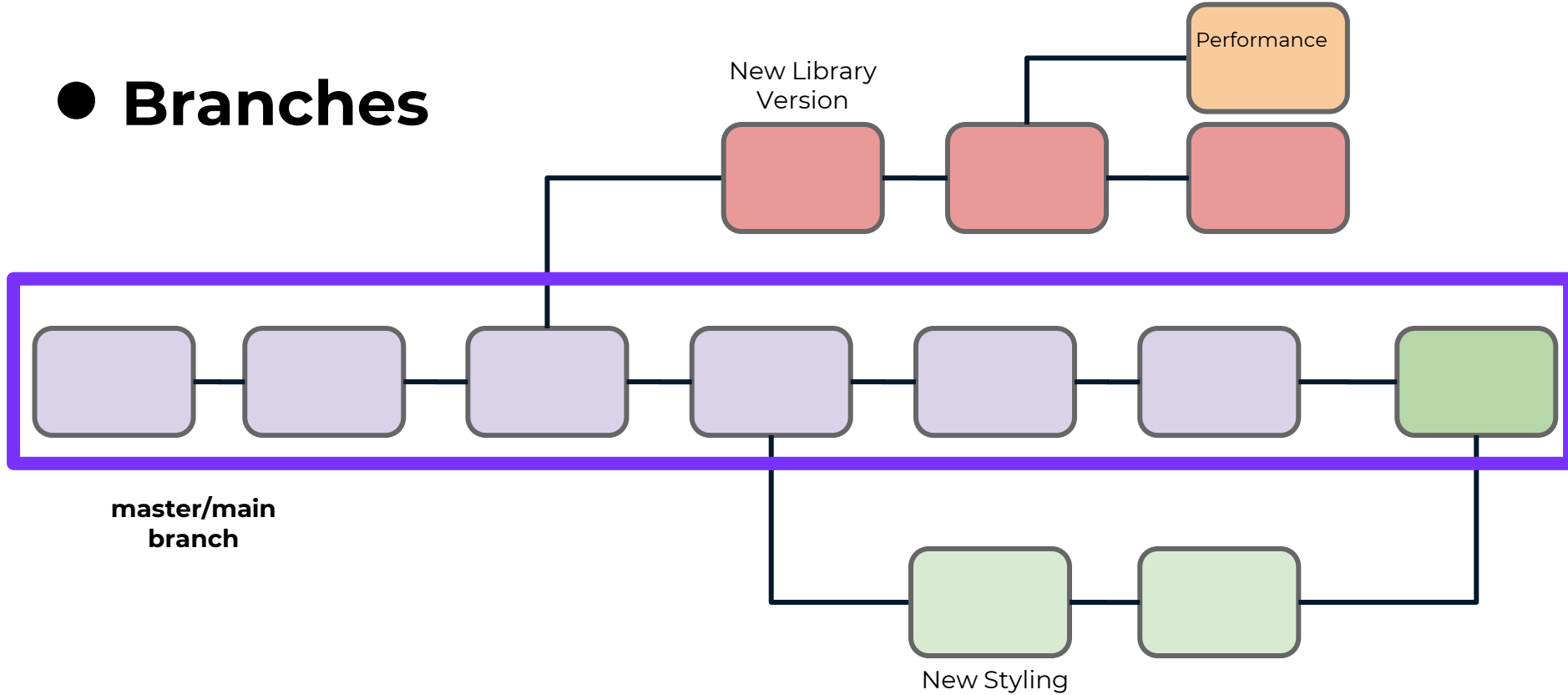
Week 3 Branches and Working with Others

● Branches



Week 3 Branches and Working with Others

● Branches



Week 3 Branches and Working with Others

● Branches

- Upon creating a new repo with **git init** you create a new branch called the **master branch** (or **main branch**).
- This is a branch just like any other, but it's simply the first one created.
 - *Should code pushed to master branch always be in working condition?*

Week 3 Branches and Working with Others

● **Branches**

- While organizations and developers often treat this master branch as the official branch for things like deployment, this is not a requirement.
- You can use any branch for code deployment or code that's actually “in-use”.

Week 3 Branches and Working with Others

● Branches

○ Master vs. Main

- As we've discussed previously, GitHub has changed the nomenclature for this initial branch to be **main branch** while Git is still using **master branch** (but this may change in the future).
- You can also rename any branch (**trunk branch**).

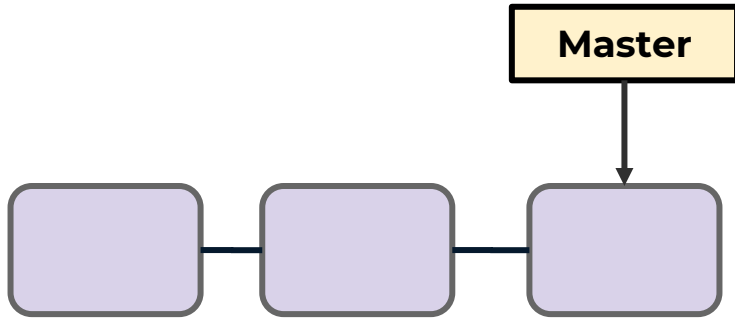
Week 3 Branches and Working with Others

● **Creating a New Branch**

- Before we conclude, let's quickly go into more detail about what happens when first create a new branch.
- Branches are just pointers to commits.
- When you create a branch, all Git needs to do is create a new pointer, it doesn't change the repository in any other way.

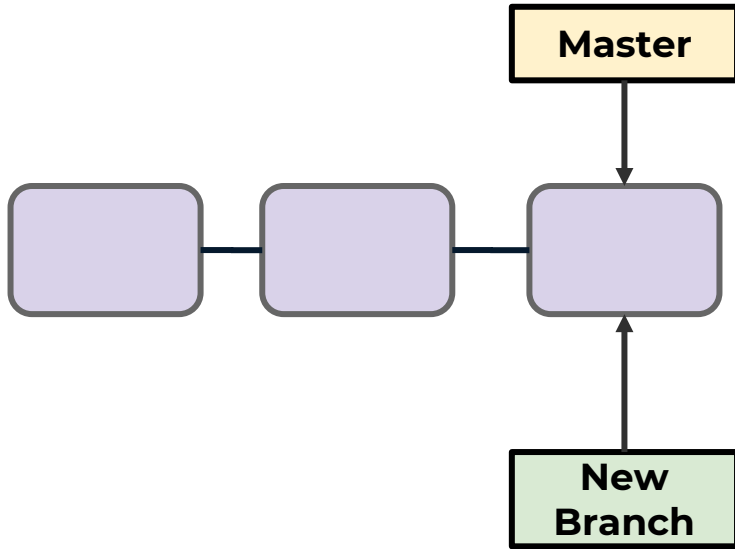
Week 3 Branches and Working with Others

- **Creating a New Branch**



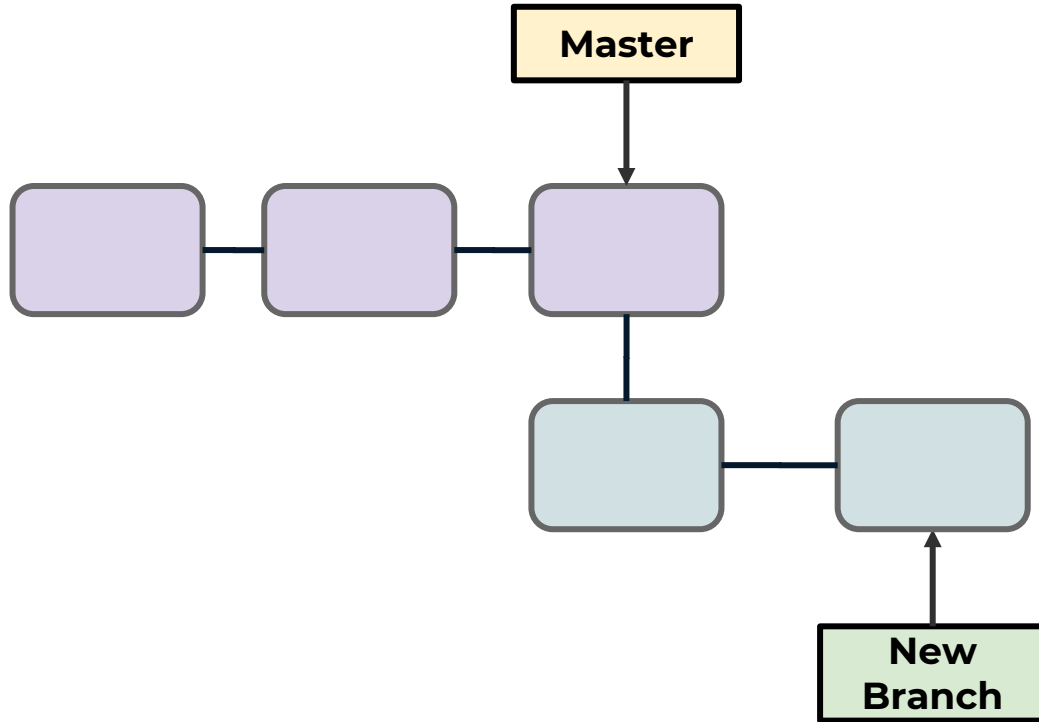
Week 3 Branches and Working with Others

- **Creating a New Branch**



Week 3 Branches and Working with Others

● Creating a New Branch



Week 3 Branches and Working with Others

- Now that we've seen how branches point to commits, we need to learn about HEAD.
- HEAD will help us understand what we are currently “viewing” or where we are “located” in regards to branches and commits.

Week 3 Branches and Working with Others

- **Up Next:**

- We'll explore and visualize specific actions and commands related to branches, including **HEAD**, **git checkout**, **git branch**, **git switch**, and more.

Week 3

Understanding HEAD

Week 3 Branches and Working with Others

● Branches

- As we work more with branches, you will probably notice a term show up during your commits: **HEAD**.
- When viewing the most recent commit using **git log** you may see:
 - **commit 05as..3e2 (HEAD -> master)**

Week 3 Branches and Working with Others

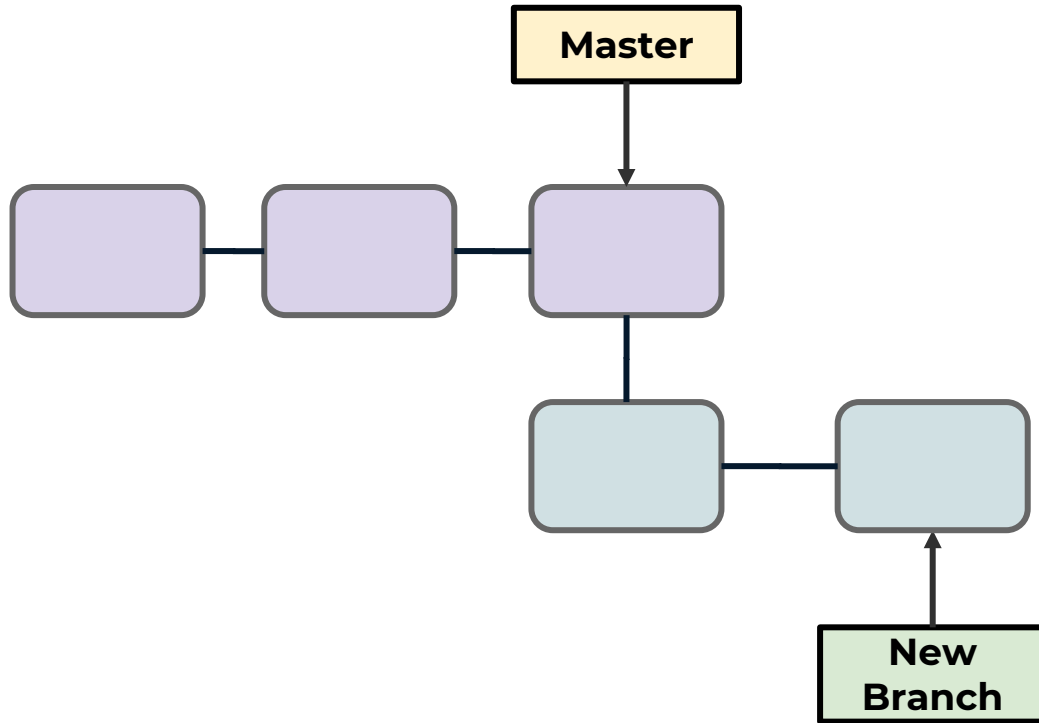
- **HEAD**

- In all of our examples so far, HEAD has always been pointing to the most recent commit in the master branch.

- **HEAD -> master**

Week 3 Branches and Working with Others

- Recall we have branch points (references)



Week 3 Branches and Working with Others

● **Branches and Commits**

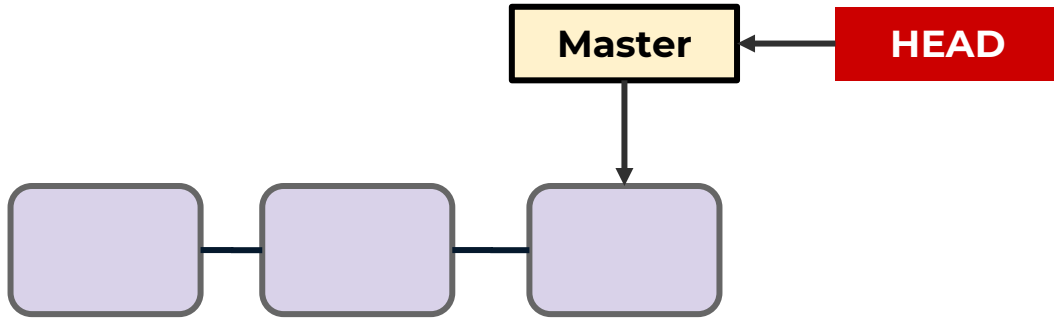
- Git stores a branch as a reference to a commit.
- In this sense, a branch represents the tip of a series of commits—it's not a container for commits.
- The history for a branch is extrapolated through the commit relationships.

Week 3 Branches and Working with Others

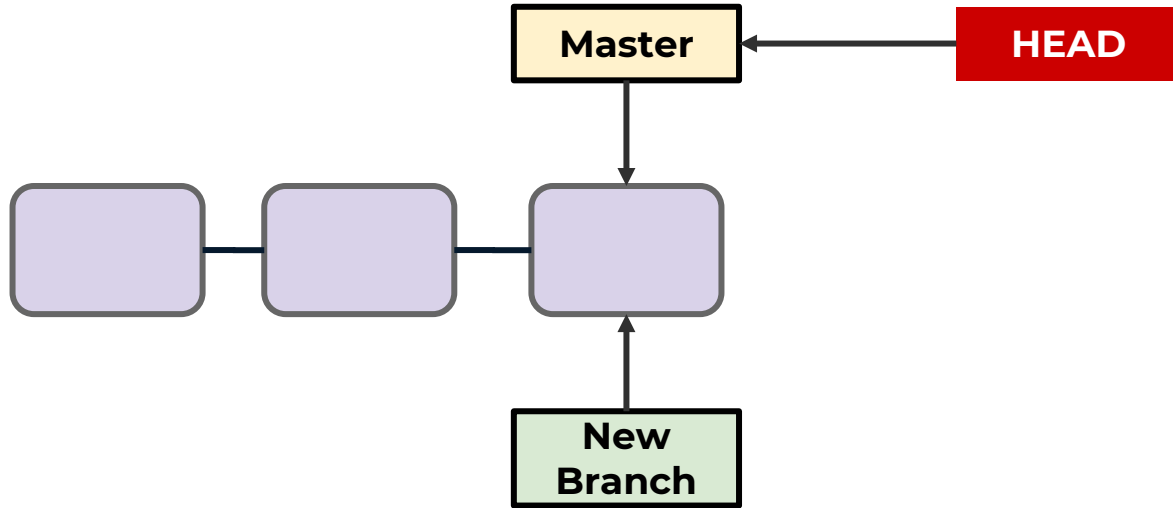
● HEAD

- A HEAD is simply a reference to a commit object.
- We can think of HEAD as pointing to a specific commit in a branch that we are currently viewing.

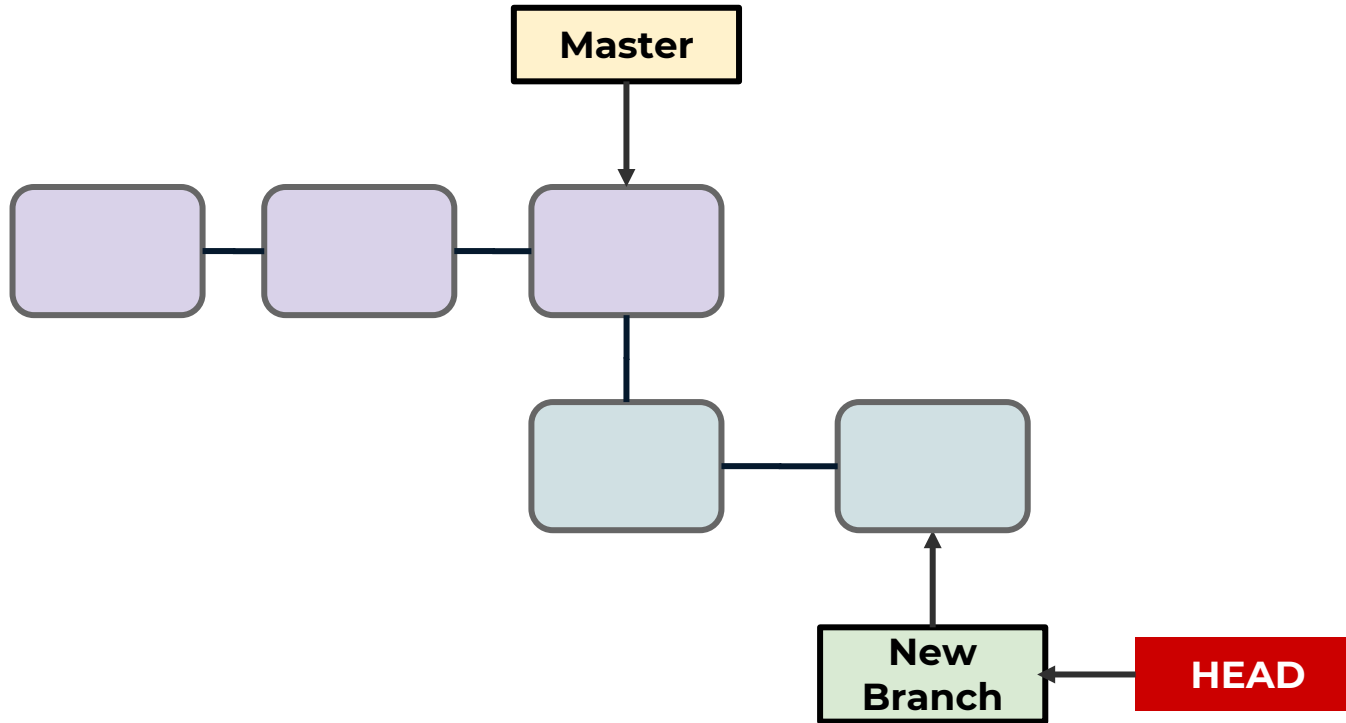
Week 3 Branches and Working with Others



Week 3 Branches and Working with Others



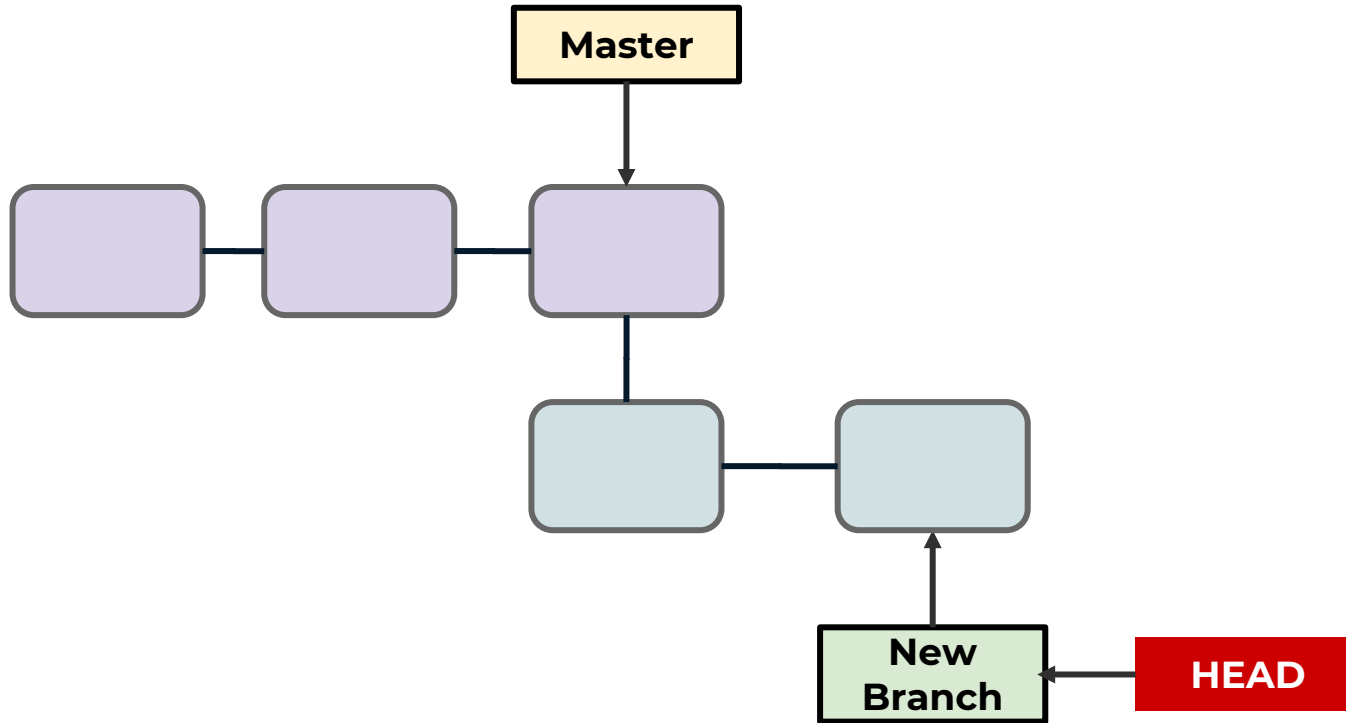
Week 3 Branches and Working with Others



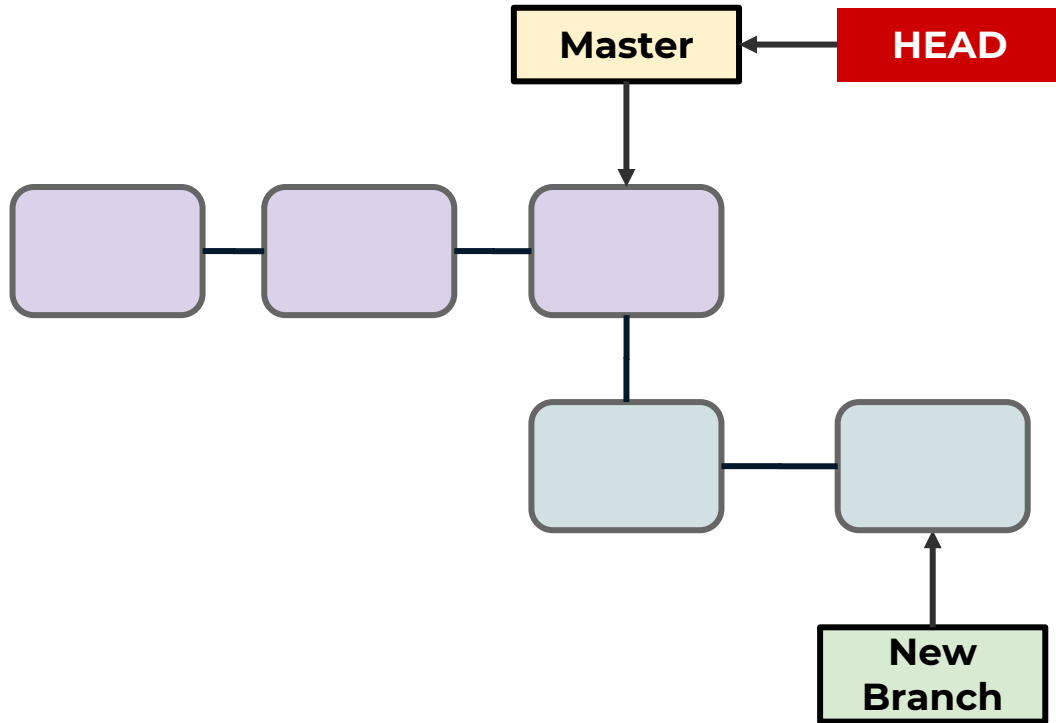
Week 3 Branches and Working with Others

- We can think of these branches as just references to a commit.
- Using HEAD tells us which branch reference we are currently “checking out”.
- We can always switch back out HEAD to some other branch (which is a pointer to a commit reference).

Week 3 Branches and Working with Others



Week 3 Branches and Working with Others



Week 3 Branches and Working with Others

- **Up Next:**

- Now that we understand the theory behind branches and HEAD, let's begin to explore the actual commands that let us create branches and navigate between them.

Week 3

Git Branch Commands

Week 3 Branches and Working with Others

● Git Branch Commands

- Create a New Repo
- Add File
- Create a New Branch
 - **git branch <branch_name>**
- Report Branches
 - **git branch**
- Switch Branches
 - **git switch or git checkout**

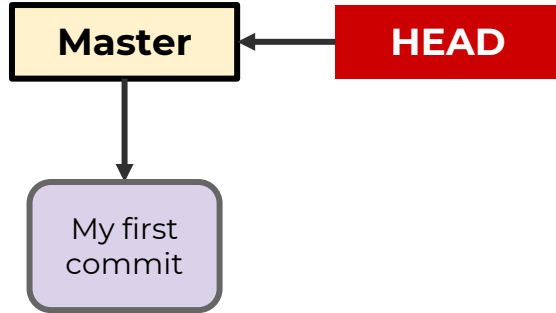
Week 3 Branches and Working with Others

● Git Branch Commands

- Add and Commit Changes on New Branch
- Use **git log** and **git switch** to explore differences between branches.

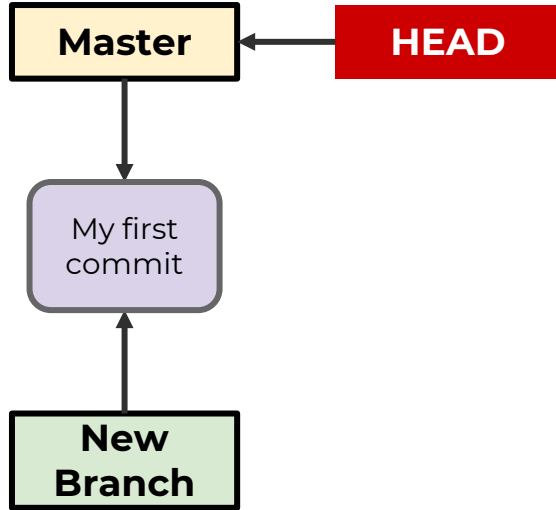
Week 3 Branches and Working with Others

- **git init, git add, git commit**



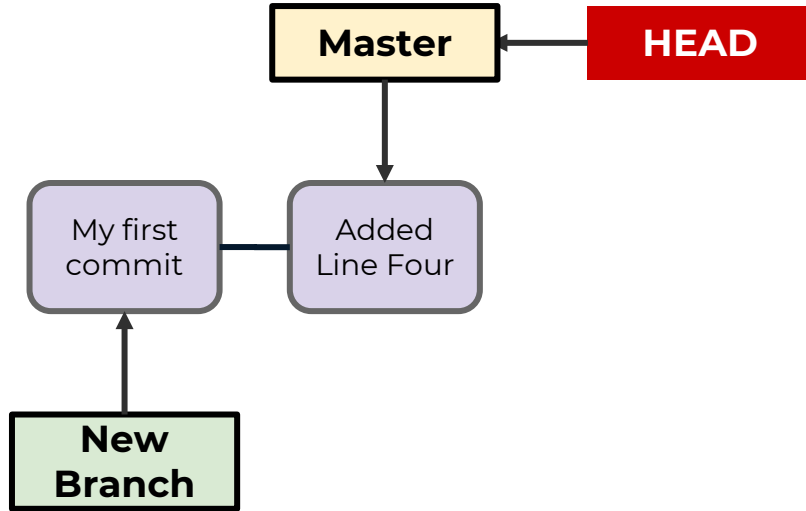
Week 3 Branches and Working with Others

● `git branch new_branch`

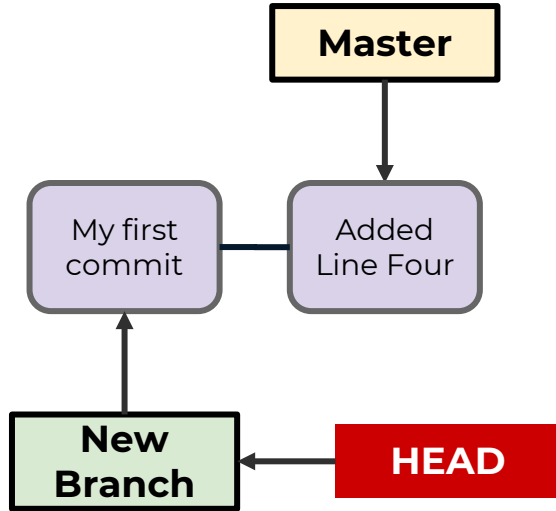


Week 3 Branches and Working with Others

- **git add, git commit, git log**



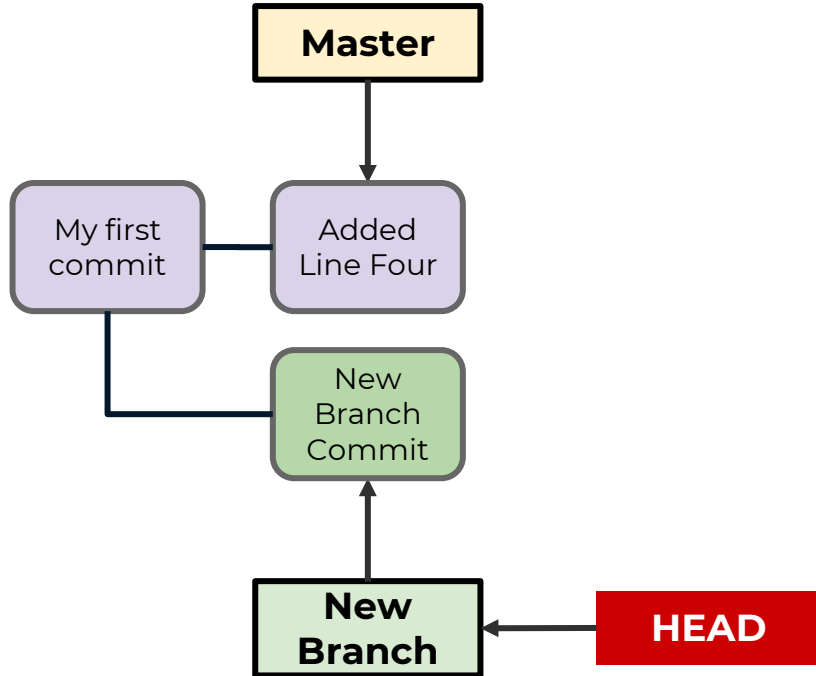
Week 3 Branches and Working with Others



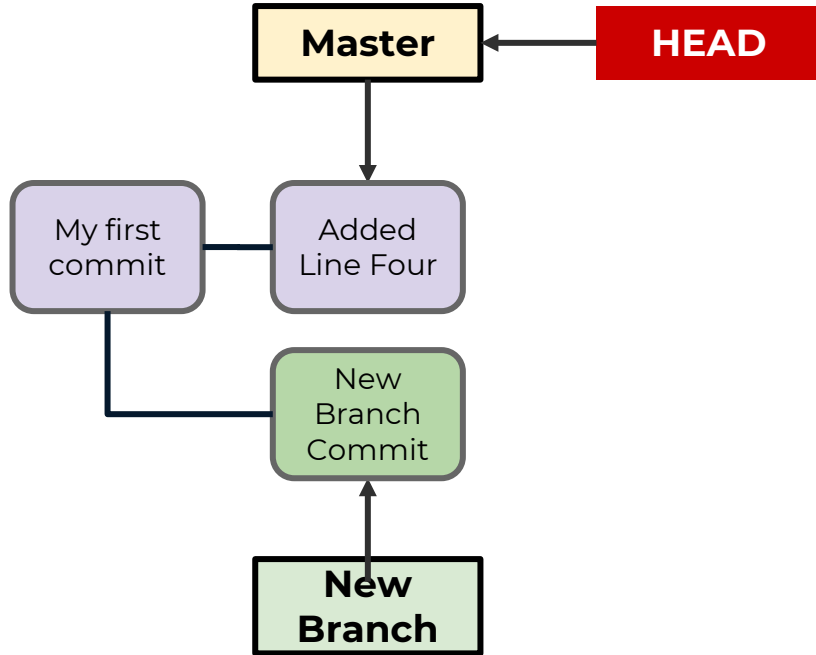
- **git switch new_branch**
or git checkout new_branch

Week 3 Branches and Working with Others

- **git add , git commit, git log**



Week 3 Branches and Working with Others



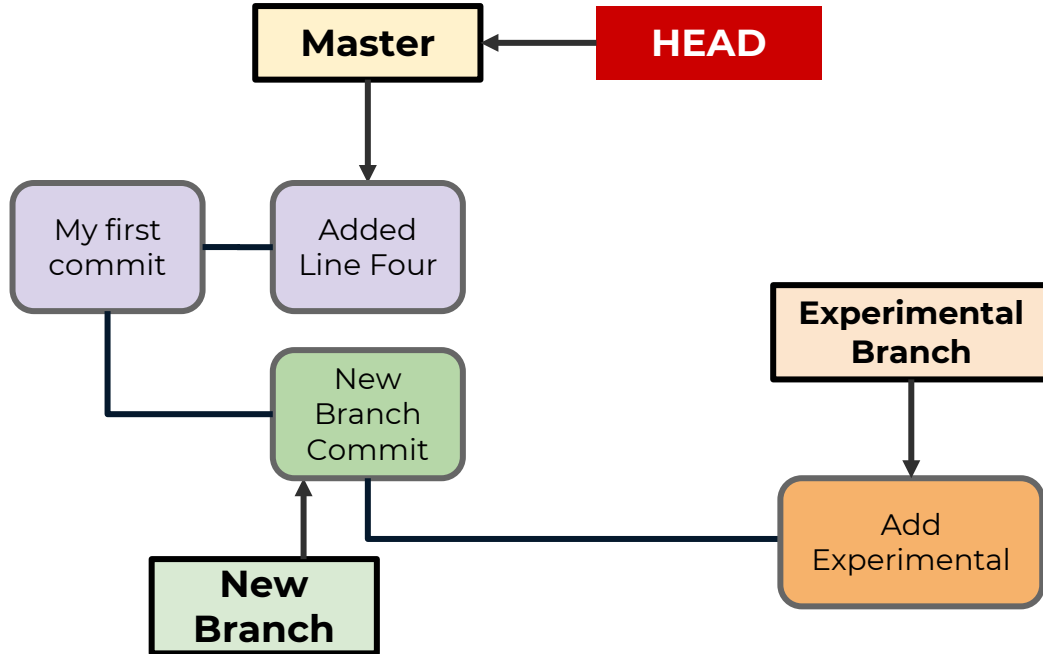
- **git switch master**
or git checkout master

Week 3

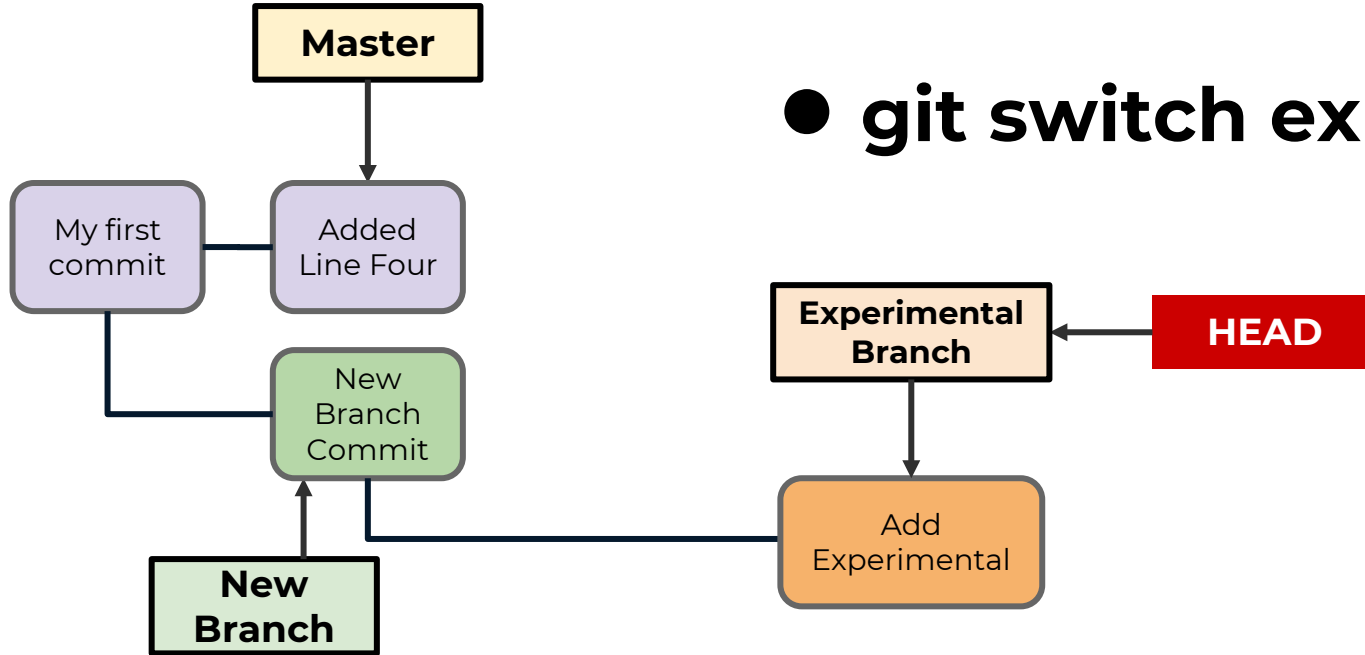
Delete and Rename Branches

Week 3 Branches and Working with Others

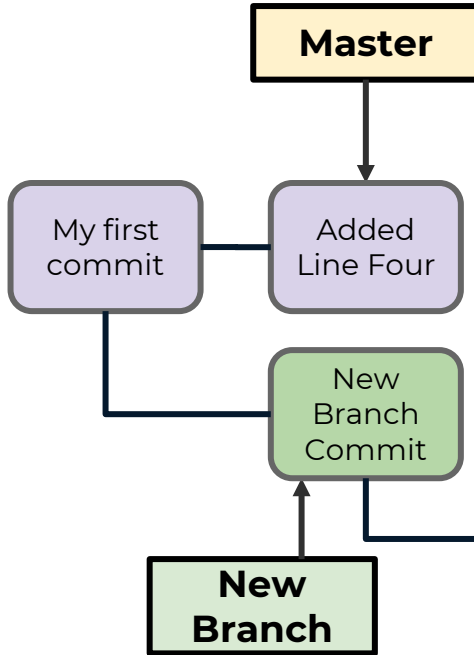
● Previously:



Week 3 Branches and Working with Others

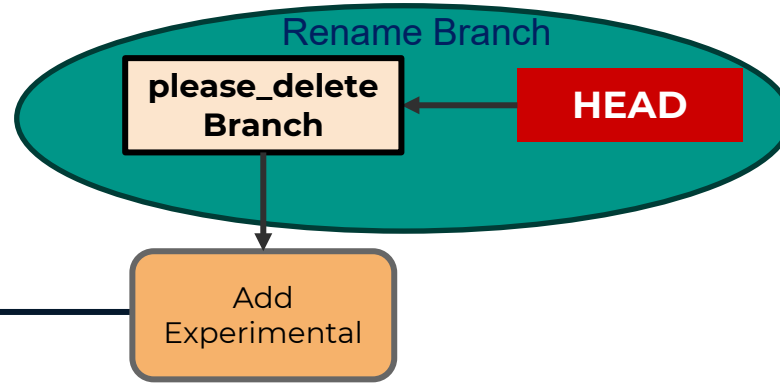


- **git switch experimental**

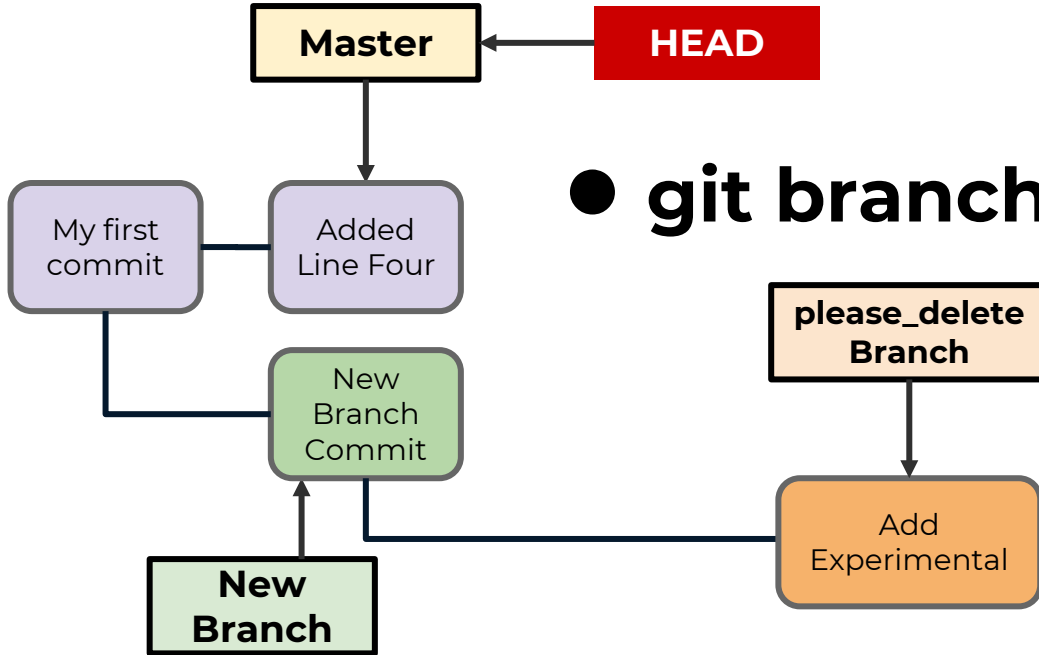


Rename branch

- `git branch -m please_delete`



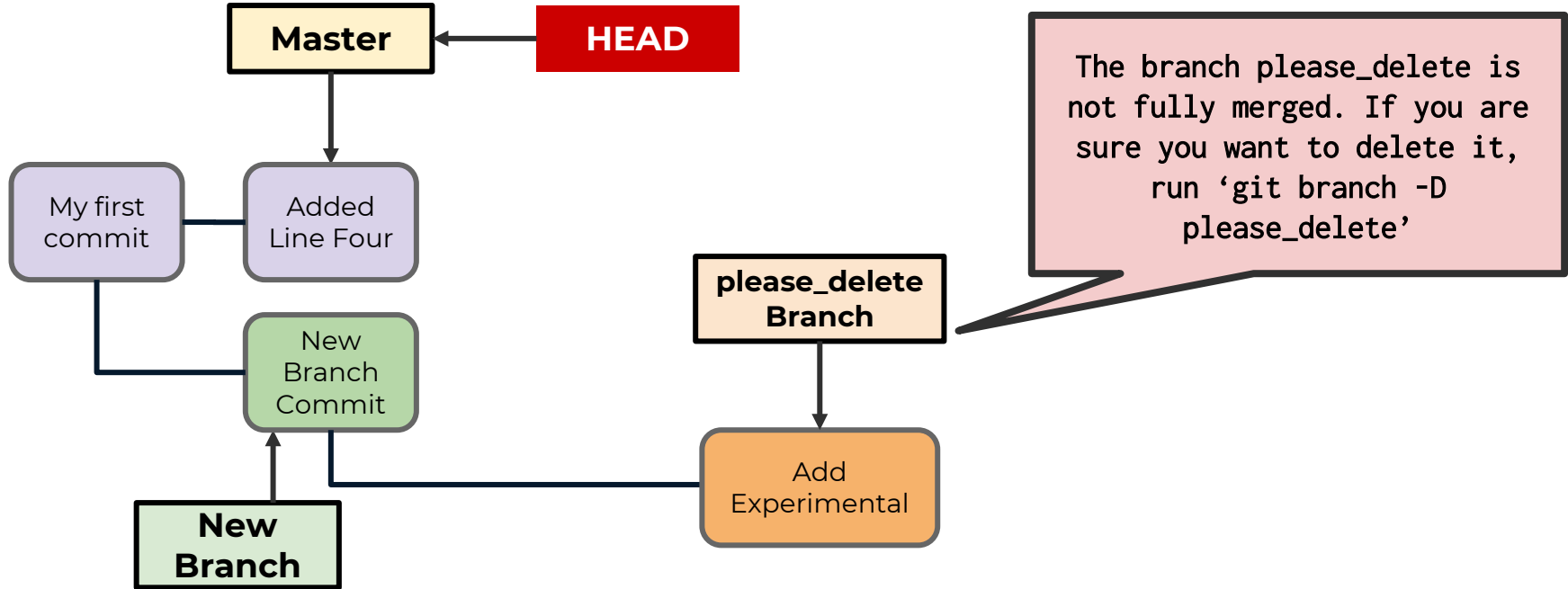
Week 3 Branches and Working with Others



- **git branch -d please_delete**

Week 3 Branches and Working with Others

● `git branch -d please_delete`



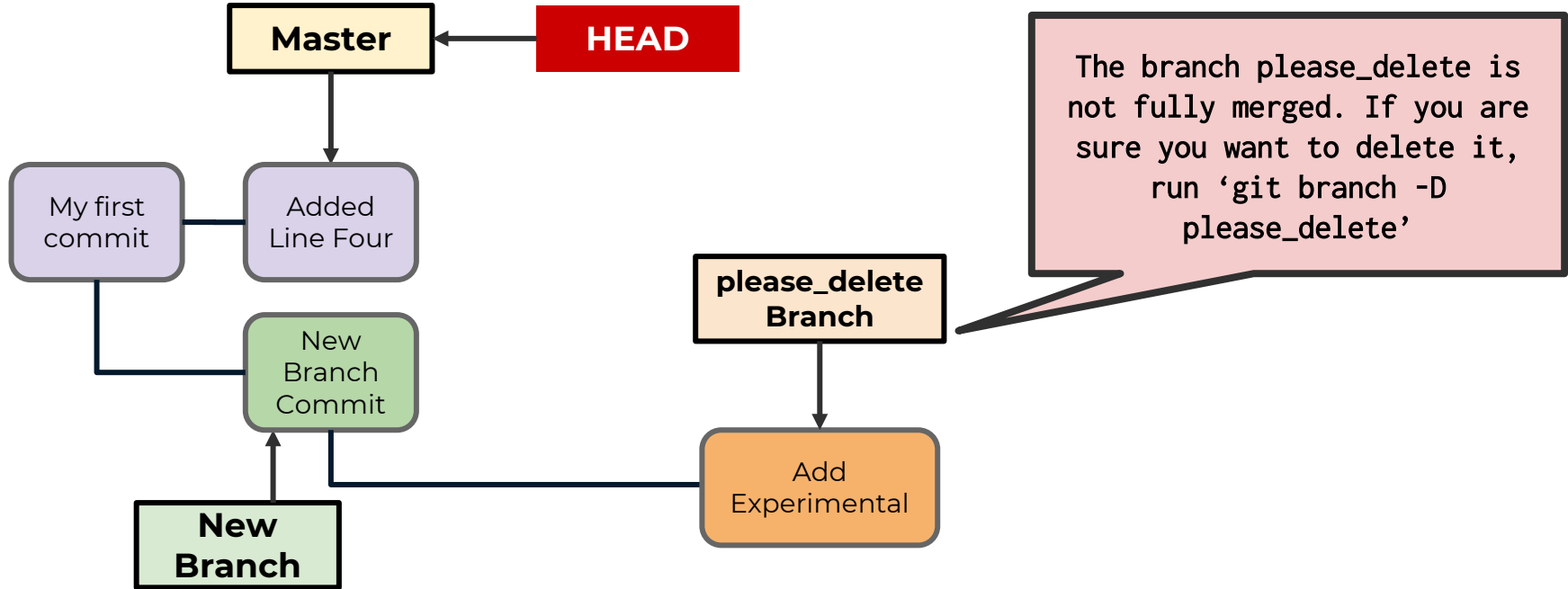
Week 3 Branches and Working with Others

- **Deleting a Branch**

- **git branch -d branch_to_delete_name**
 - You can not delete a branch you are checked out at.
 - You also will get a warning if the branch is not merged.
 - You can confirm you want to do this anyways with **-D**

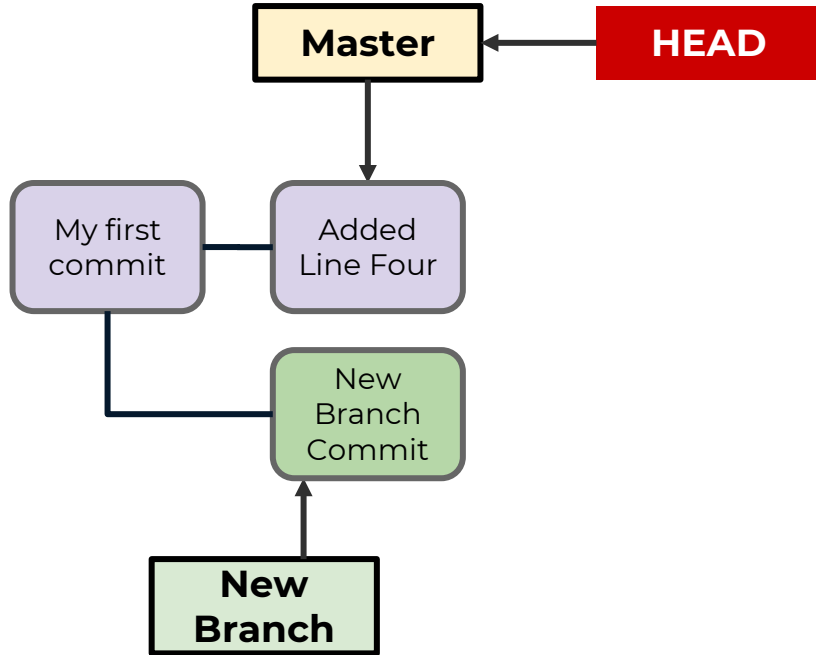
Week 3 Branches and Working with Others

● git branch -D please_delete



Week 3 Branches and Working with Others

- **git branch -D please_delete**



Week 3

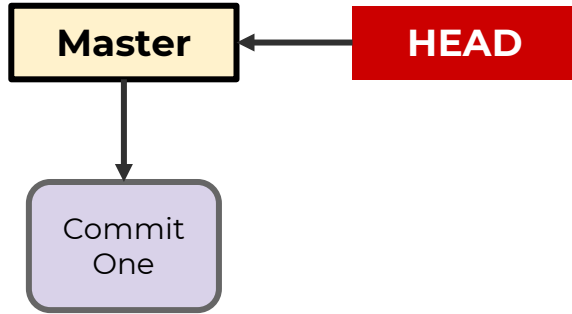
Merging Branches and Conflicts

Week 3 Branches and Working with Others

- Now that we understand creating new branches, let's shift focus to merging branches back together.
- Let's explore a simple type of merge, where a new branch is created, but the original branch it stemmed from has no additional commits.
 - This is known as a “fast-forward” merge

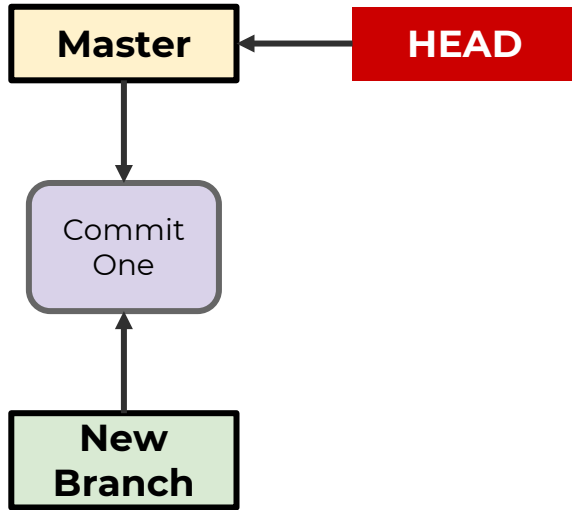
Week 3 Branches and Working with Others

● “Fast Forward” Merge



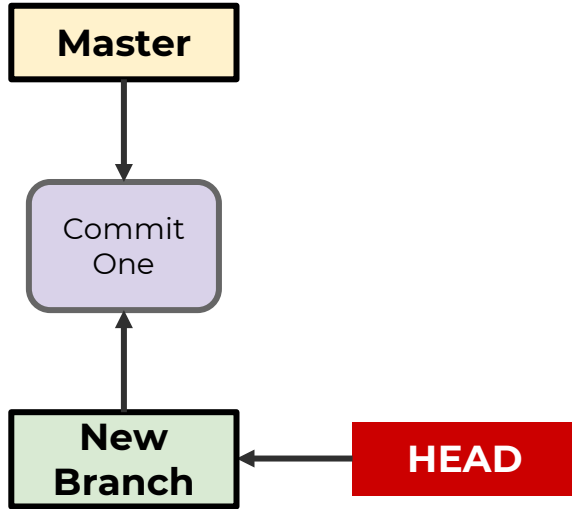
Week 3 Branches and Working with Others

● “Fast Forward” Merge



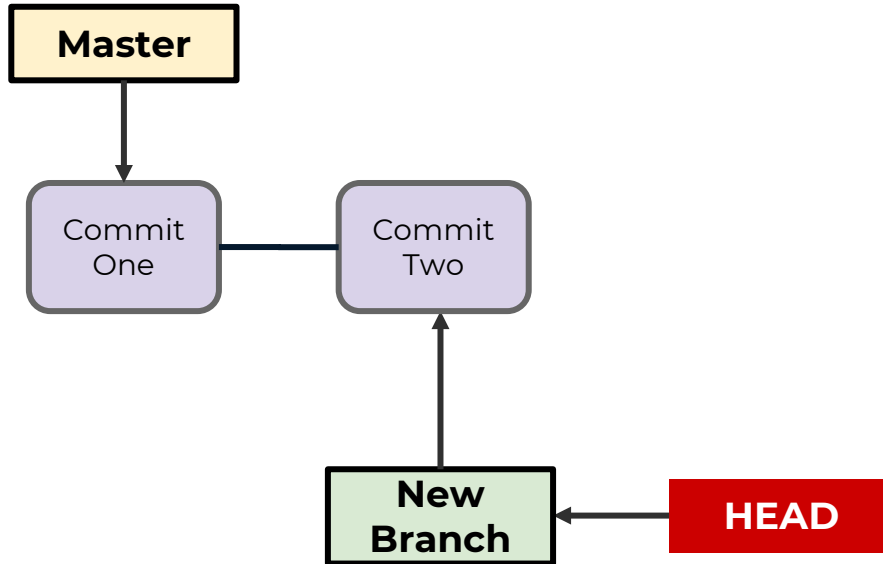
Week 3 Branches and Working with Others

● “Fast Forward” Merge



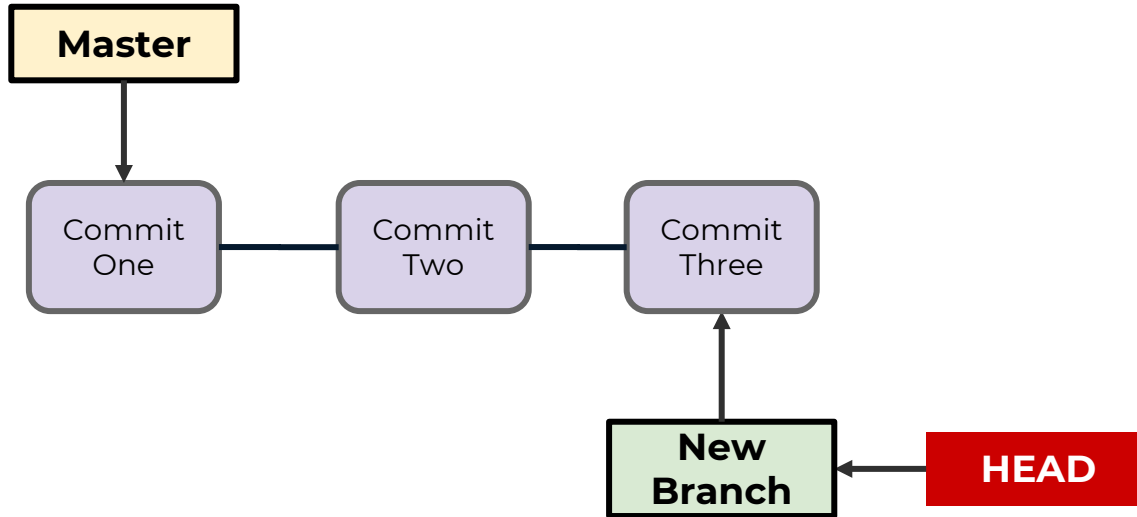
Week 3 Branches and Working with Others

● “Fast Forward” Merge



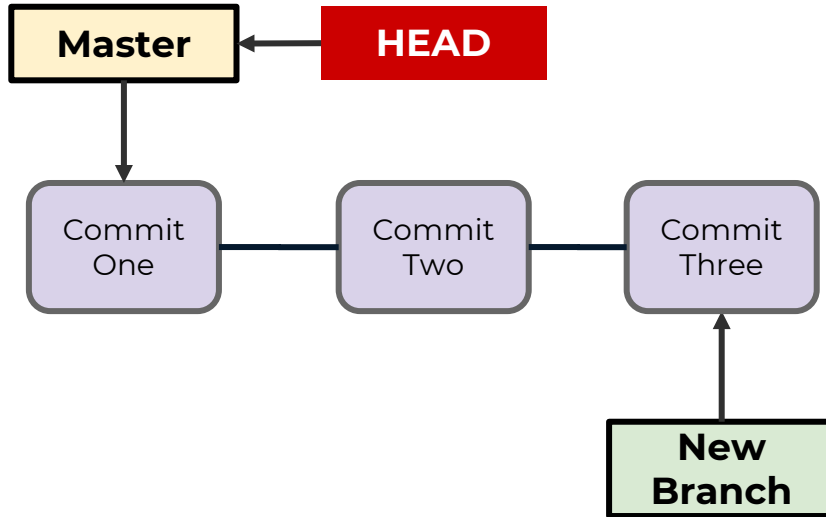
Week 3 Branches and Working with Others

● “Fast Forward” Merge



Week 3 Branches and Working with Others

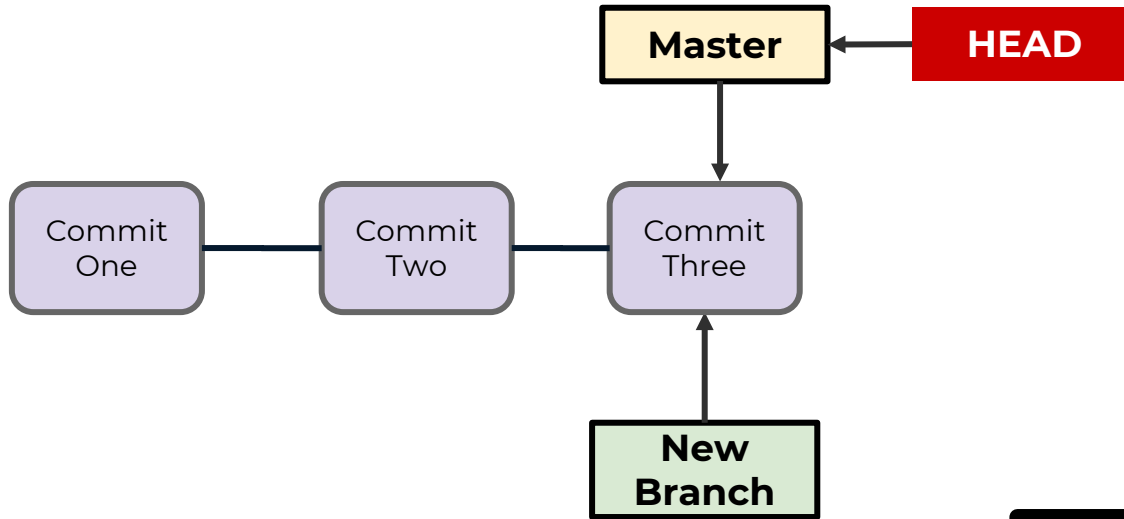
● “Fast Forward” Merge



```
>> git switch master
```

Week 3 Branches and Working with Others

● “Fast Forward” Merge



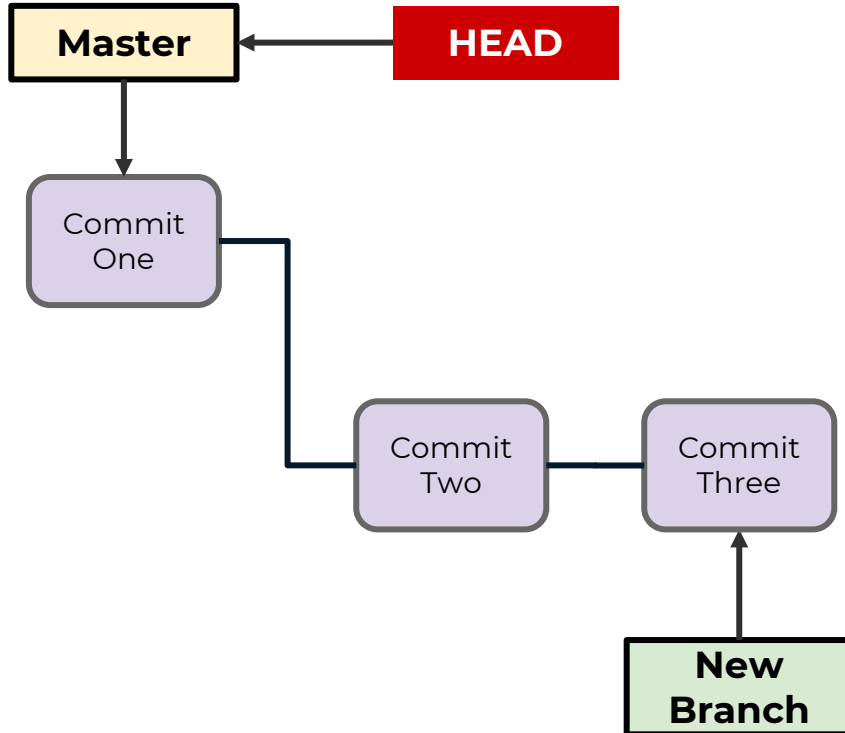
```
>> git merge new_branch
```

Week 3 Branches and Working with Others

- Now let's explore what happens for a merge where we have different commits in the branches.

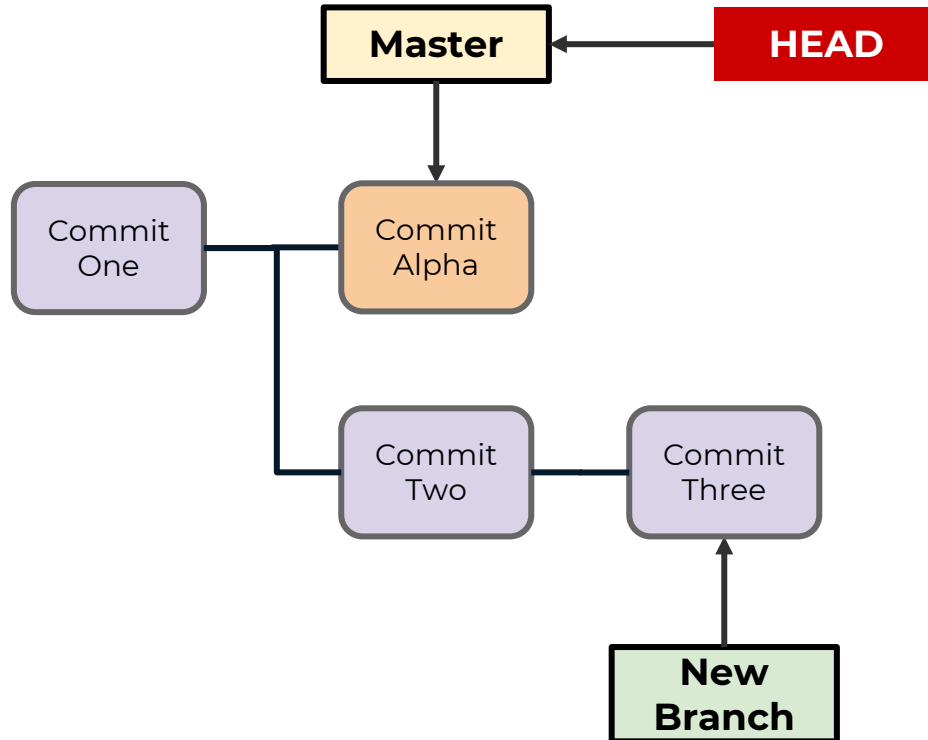
Week 3 Branches and Working with Others

● Git Merge



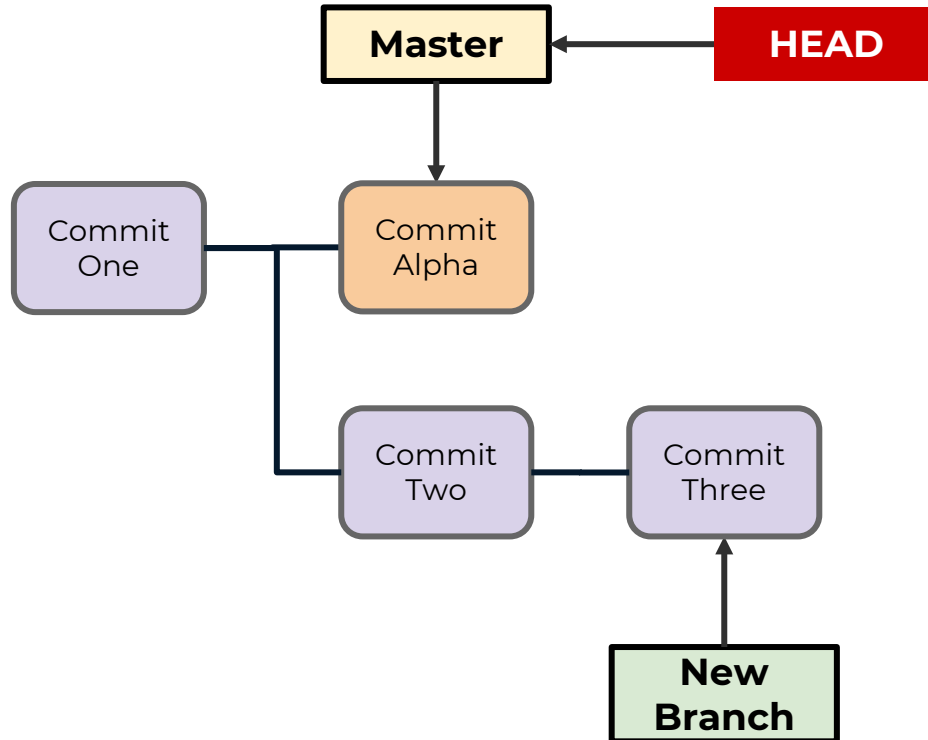
Week 3 Branches and Working with Others

● Git Merge



Week 3 Branches and Working with Others

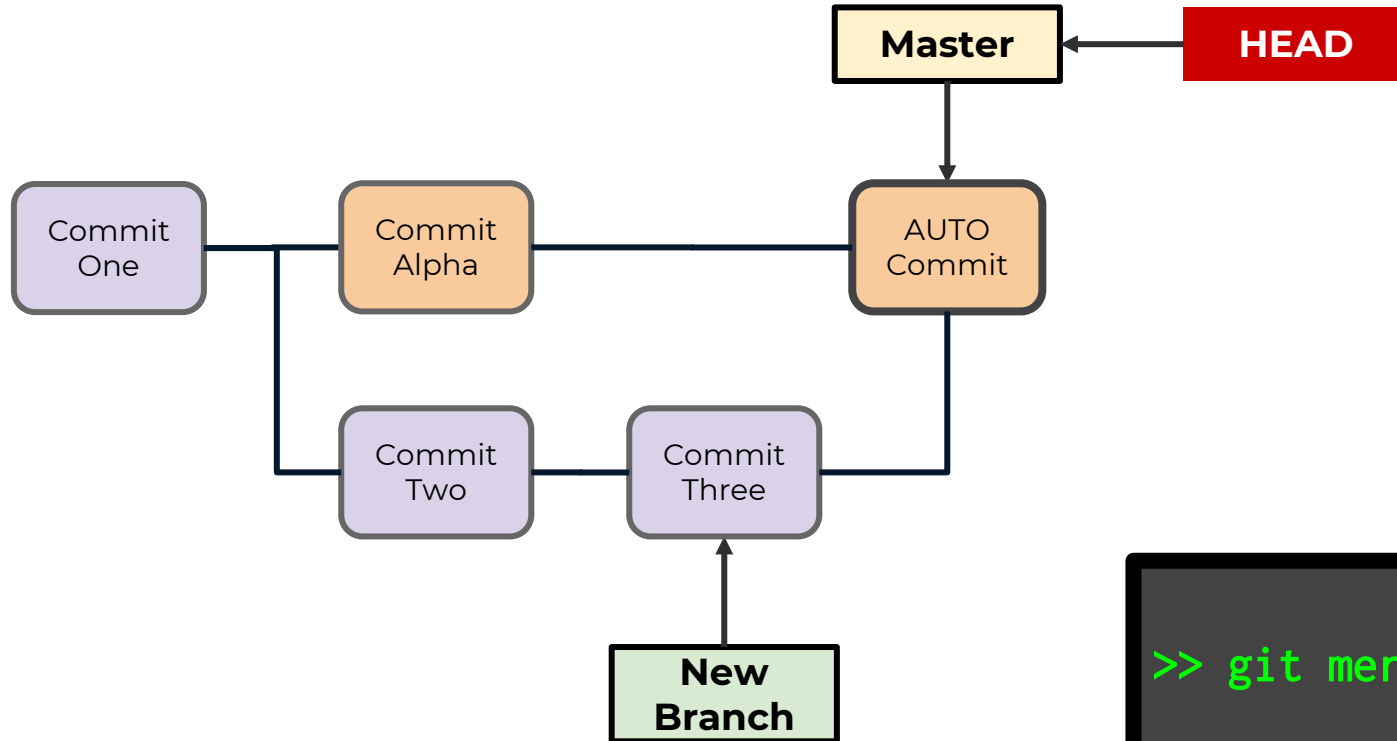
● Git Merge



```
>> git merge new_branch
```

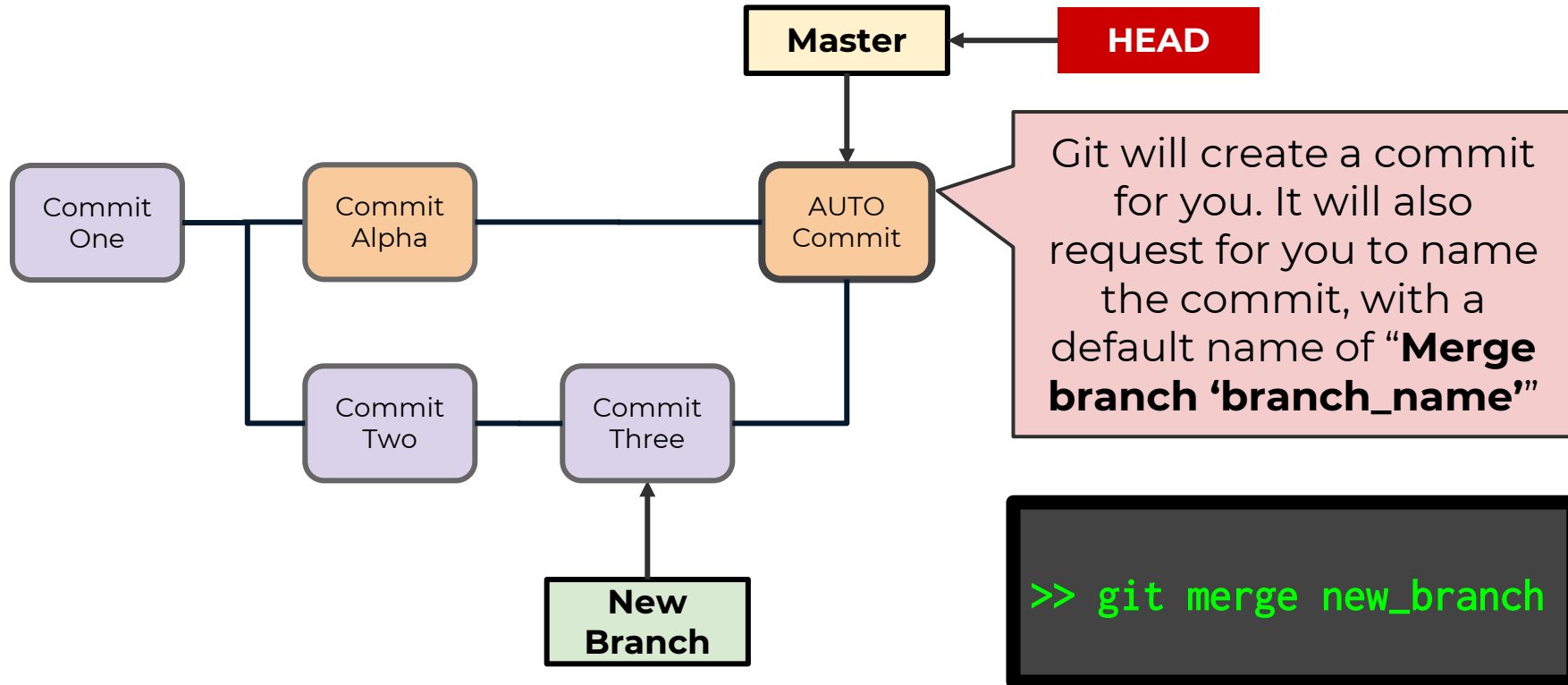
Week 3 Branches and Working with Others

● Git Merge



Week 3 Branches and Working with Others

● Git Merge



Week 3 Branches and Working with Others

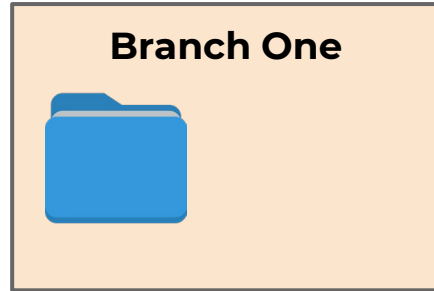
- Git creates the new commit for us, and will attempt the merge.
- Sometimes there are no conflicts, for example:
 - The branch only focused on files not in the receiving branch, thus the merge simply adds the new files to the receiving branch.

Week 3 Branches and Working with Others

- Git will try to automatically create the merge, and can do so in cases where no information from a specific branch would be lost, for example:

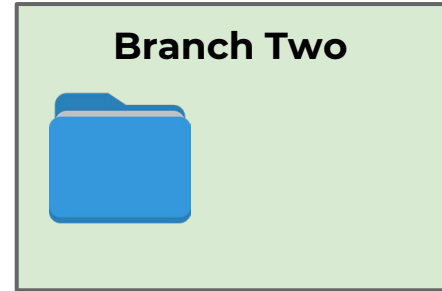
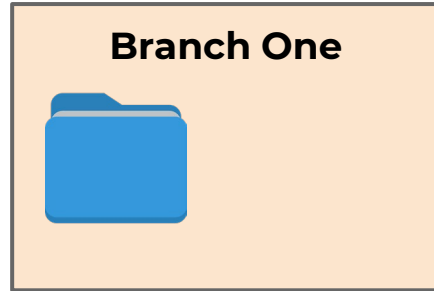
Week 3 Branches and Working with Others

- Git will try to automatically create the merge, and can do so in cases where no information from a specific branch would be lost, for example:



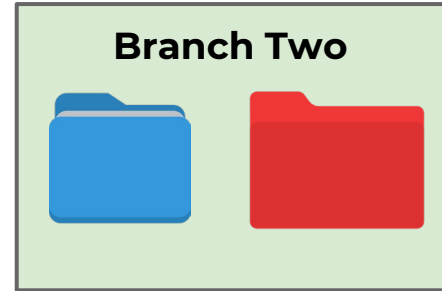
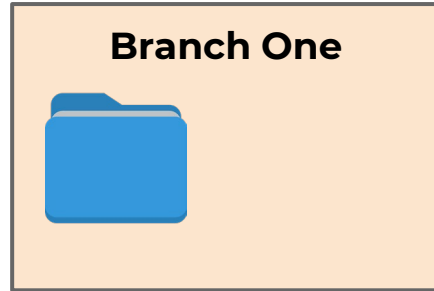
Week 3 Branches and Working with Others

- Git will try to automatically create the merge, and can do so in cases where no information from a specific branch would be lost, for example:



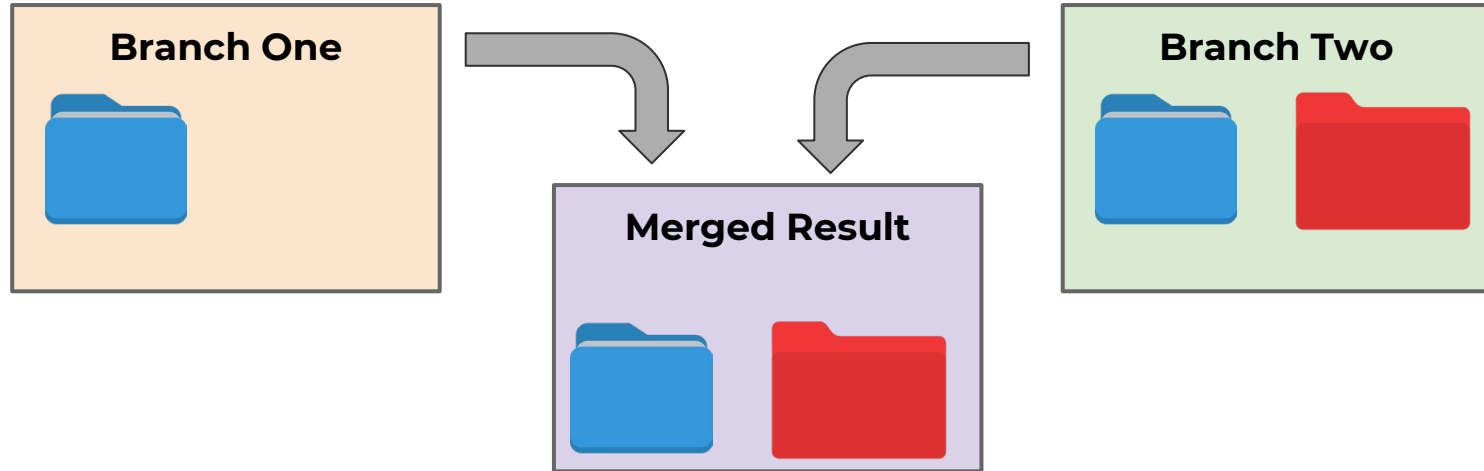
Week 3 Branches and Working with Others

- Git will try to automatically create the merge, and can do so in cases where no information from a specific branch would be lost, for example:



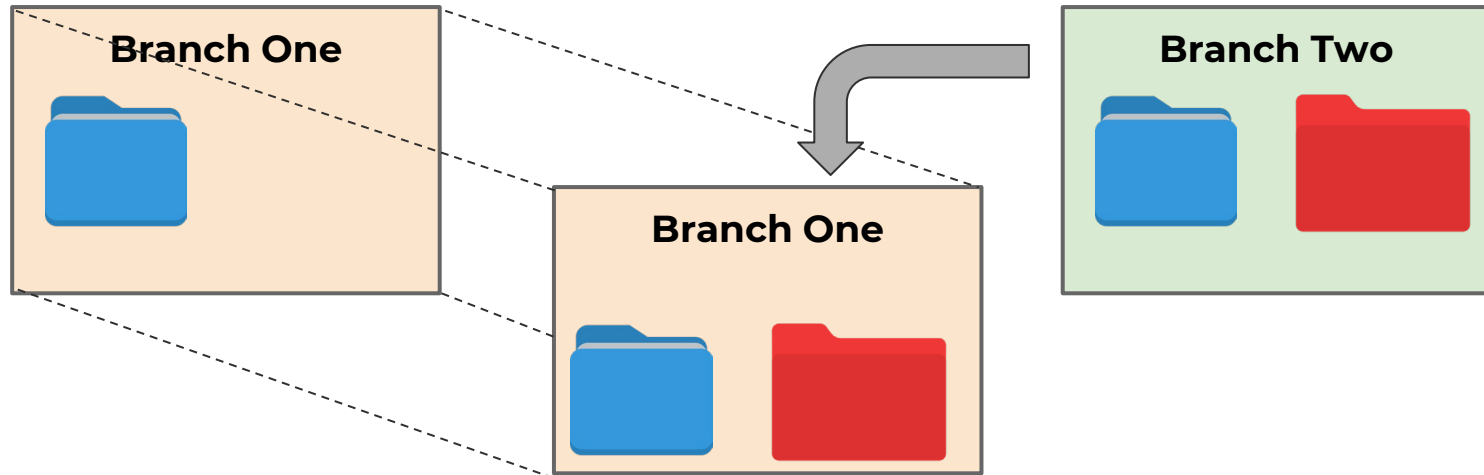
Week 3 Branches and Working with Others

- Git will try to automatically create the merge, and can do so in cases where no information from a specific branch would be lost, for example:



Week 3 Branches and Working with Others

- Git will try to automatically create the merge, and can do so in cases where no information from a specific branch would be lost, for example:



GIT DEEP DIVE

git diff

อ่านความเปลี่ยนแปลงให้ขาด — ตั้งแต่ไฟล์ที่เพิ่งแก้ไขไปจนถึงการ
เทียบข้าม branch

```
@@ -1,3 +1,4 @@
```

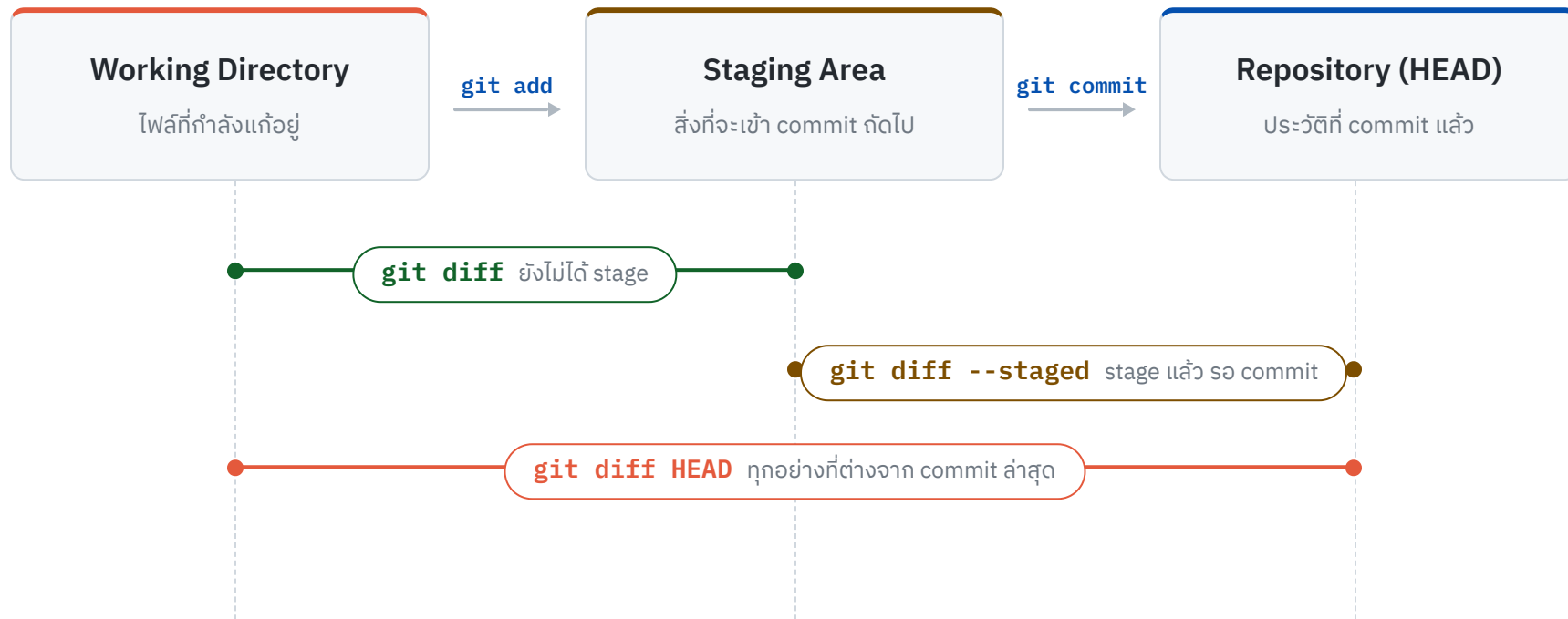
ความรู้ git diff ของทีม

- ใ้แบบใดๆ ดู output ผ่านๆ

+ อ่านทุกบรรทัดออก และเลือกคำสั่งถูกสถานการณ์

สามพื้นที่ของ Git และ diff

diff คือการเทียบ snapshot สองจุด — คำสั่งแต่ละแบบแค่เลือก "จุด" ต่างกัน



อ่าน diff output ให้เป็น

```
diff --git a/app.js b/app.js①
index 3f5a2c1..8d4e9b7 100644②
--- a/app.js③
+++ b/app.js
@@ -12,7 +12,8 @@ function login(user) {④
     if (!user) return null;
-   const token = sign(user.id);
+   const token = sign(user.id, SECRET);
+   audit.log('login', user.id);
   return token;
```

① a/ = ฝั่งเก่า, b/ = ฝั่งใหม่ ของไฟล์ที่เทียบ

② blob hash ก่อน/หลัง และ file mode

③ - มาจากไฟล์เก่า, + มาจากไฟล์ใหม่

④ hunk header — บอกตำแหน่งบรรทัด (สไลด์ถัดไป)

บรรทัดที่ขึ้นต้นด้วย space คือ context — ไม่เปลี่ยนแปลง แสดงไว้ให้เห็นบริบทรอบๆ

Hunk header: @@ บอกอะไร

```
@@ -12,7 +12,8 @@ function login(user) {
```

-12,7

ฝั่งเก่า: hunk นี้เริ่มบรรทัดที่ **12** ถึงพื้นที่ **7** บรรทัด

+12,8

ฝั่งใหม่: เริ่มบรรทัดที่ **12** ยาว **8** บรรทัด —
เพิ่มสุทธิ 1 บรรทัด

function ...

ข้อความหลัง @@ คือ scope ที่ git เดาให้
— ช่วยบอกว่า hunk อยู่ใน function ไหน

ตัวอย่าง: unstaged vs staged

แก้ app.js หนึ่งบรรทัด แล้วดูว่าแต่ละคำสั่งเห็นอะไร

ก่อน git add

```
$ git diff
@@ -12,1 +12,1 @@
-const token = sign(user.id);
+const token = sign(user.id, SECRET);

$ git diff --staged
(ว่าง - ยังไม่มีอะไรใน stage)
```

หลัง git add app.js

```
$ git diff
(ว่าง - การแก้ไขไม่ stage แล้ว)

$ git diff --staged
@@ -12,1 +12,1 @@
-const token = sign(user.id);
+const token = sign(user.id, SECRET);
```

git diff HEAD เห็นทั้งสองกรณีเสมอ · --cached คือชื่อเก่าของ --staged

เปรียบเทียบ commits

รูปแบบ: `git diff <ฐาน> <ปลายทาง>` – ลำดับกำหนดว่าจะไรคือ – อะไรคือ +

```
git diff 3f5a2c1 8d4e9b7
```

เทียบ commit สองตัวด้วย hash

```
git diff HEAD~3 HEAD
```

สาม commit ล่าสุดเปลี่ยนอะไรรวมกันบ้าง

```
git diff HEAD~1
```

ใส่ตัวเดียว = เทียบ commit นั้นกับ working directory

จำง่ายๆ: อ่านว่า "จาก **ฐาน** ไปเป็น **ปลายทาง** ต้องเปลี่ยนอะไรบ้าง"

two-dot vs three-dot

feature แยกจาก main ที่จุด **B** (merge-base) แล้วต่างคนต่างเดิน

```
main      A — B — C — D
           |
           L — E — F   feature
```

`git diff main..feature`

เทียบปลายทั้งสอง: **D** vs **F** — เหมือนเขียน main feature
เฉยๆ

งานใหม่บน main จะโผล่มาเป็นบรรทัดลบ

`git diff main...feature`

เทียบจาก merge-base: **B** vs **F** — เห็นเฉพาะงานของ feature

คือ diff ที่ Pull Request แสดงให้เราดู

⚠ ความหมายนี้ **ตรงข้ามกับ git log** — ใน log สอง dot คือ "commit ที่ feature มีแต่ main ไม่มี"

เปรียบเทียบ branches และ tags

อะไรที่ชี้ไปหา commit ได้ ใช้เป็น argument ได้หมด — branch, tag, remote ref

```
git diff main feature/checkout
```

สอง branch ต่างกันตรงไหน ณ ตอนนี้

```
git diff main...feature/checkout
```

เฉพาะงานของ feature — มุมมองเดียวกับ PR

```
git diff origin/main HEAD
```

ก่อน push — เราทำล้งจะส่งอะไรขึ้นไปบ้าง

อย่าลืม `git fetch` ก่อน ถ้าจะเทียบกับ `origin/*` ให้เป็นข้อมูลล่าสุด